O‘QUV QO‘LLANMA · TEST VA IMTIHONLARGA TAYYORLANISH

Informatika: C da dasturlash 4–7 haftalar

Barcha qoidalar, savollar va vazifalar ma’ruzalardan, laboratoriyalardan va bug hunt-lardan kurs — model javoblar, kod tahlili va har hafta uchun yechimlar bilan haftalar: ma’lumot turlari, boshqaruv konstruksiyalari, funksiyalar, strukturalar, ko‘rsatkichlar, satrlar va rekursiya.

Computer Science (C Dasturlash) · 2026-yil kuz semestri
Toshkent texnologiya universiteti

SAVOLLAR · TESTLAR · KOD · QOIDALAR

OKTABR 2026

Tarkibi

1. Kurs xaritasi, manbalar va qo‘llanma qanday foydalanish . . . 3

1.1. 4–7-haftalar va kurs fayllari . . . 3

2. 4-hafta — Ma’lumot turlari va operatorlar (Data Types and Operators) . . . 4

2.1. Asosiy g‘oyalar (4-haftaning "barcha qoidalari") . . . 4

2.2. 4-hafta savollari — ma’ruza va laboratoriya savollari model javoblar bilan . . . 9

2.3. 4-haftaviy testlar . . . 11

2.4. Atamalar — 4-hafta glossariysi . . . 13

3. 5-hafta — Boshqaruv konstruksiyalari (Control Structures) . . . 13

3.1. Asosiy tushunchalar («5-hafta barcha qoidalar») . . . 14

3.2. 5-hafta savollari — dars va laboratoriya savollari model javoblar bilan . . . 19

3.3. 5-hafta testlari . . . 20

3.4. Atamalar — 5-hafta glossariysi . . . 22

4. 6-hafta — Funksiyalar va tuzilmalar (Functions and Structures) . . . 23

4.1. Asosiy tushunchalar («haftaning barcha qoidalari» 6) . . . 23

4.2. 6-hafta savollari — ma’ruza va laboratoriya savollari model javoblar bilan . . . 29

4.3. 6-hafta testlari . . . 31

4.4. Atamalar — 6-hafta glossariysi . . . 33

5. 7-hafta — Ko‘rsatkichlar, satrlar va rekursiya (Pointers, Strings, Recursion) . . . 33

5.1. Asosiy tushunchalar (7-haftaning "barcha qoidalari") . . . 34

5.2. 7-hafta savollari — ma’ruza va laboratoriya savollari model javoblar bilan . . . 40

5.3. 7-hafta testlari . . . 42

5.4. Atamalar — 7-hafta glossariysi 44

6. Bug Hunts — barcha xatolar va ularning tuzatilishi 45

6.1. Bug Hunt C 45

6.2. Bug Hunt Loops 50

6.3. Bug Hunt Pointers & Strings 54

7. Yordamchi: asosiy qoidalar va jadvallar 59

7.1. Operator ustunligi 59

7.2. printf/scanf formatlari 59

7.3. C asosiy qoidalari 60

7.4. Bir qatorda o‘n javob 61

7.5. Keyin nima 61

1. Kurs xaritasi, manbalar va qo‘llanma qanday foydalanish

Bu o‘quv qo‘llanmasi **Computer Science — C dasturlash** kursi (trk **Foundations of Computer Science, FoCS**; 2026-yil kuzgi semestr, Toshkent texnologiya universiteti) barcha fayllari asosida tuzilgan. Kurs to‘rt qatlamdan iborat: (1) haftalik ma’ruzalar — FoCS/TashTech Ma’ruzalari 4–7 (2026-yil 6, 13, 20 va 27 oktyabr) — oldingi mavzuni takrorlash va yangi mavzu; (2) Ceri, Mandrioli & Sbattella «The Art and Craft of Computing» darsligining 5–8-boblariga asoslangan laboratoriya ishlari CS50 yordamchilari (get_{double}, get_{char}, get_{string}) bilan C11-adaptatsiyasida; (3) **bug hunt** ish varaqchalari — shartli tuzilmalar va funksiyalar, sikllar, ko‘rsatkichlar va satrlar bo‘yicha «xatolarni izlash»; (4) **kofe mashinasi** loyiha fayllari (6-hafta modeli: Stock, Recipe, Cup). Qo‘llanma 4–7-haftalarning materialini qamrab oladi: ma’lumot turlari va operatorlar, boshqaruv tuzilmalari, funksiyalar va strukturalar, ko‘rsatkichlar, satrlar va rekursiyani.

Ma’ruzalar va laboratoriyalardagi har bir savol (savollar), har bir test (testlar), har bir atama (atamalar) va slaydlar, laboratoriyalar va bug hunt-lardagi har bir kod parchasi to‘liq keltirilgan va model javob yechim bilan ta’minlangan. Barcha kod misollari kurs fayllaridan so‘zma-so‘z olingan; PDF-chop etilgan ro‘yxat bo‘shliqlar bilan «zararlangan» bo‘lsa, u slayd mazmuni asosida tiklangan. O‘qituvchining asl formulalari saqlangan: «Pop quiz», «Prediction», «Exit check», «Expected».

Qo‘llanmani qanday foydalanish: (1) har haftaning «Asosiy nuqtalar» bo‘limidan boshlang — bu haftaning «barcha qoidalarini» (qoidalar) jadval bilan qisqartirib beradi; (2) ochiq savollarni (savollar) javoblarni yopib, og‘izdan aytib ko‘ring va model javoblar bilan solishtiring; (3) barcha testlarni (testlar) o‘ting — nazorat ishlarida aynan shunday formulalar qo‘llaniladi; (4) kodni qatorma-qator tahlil qiling: imtihonda «bu nima chiqaradi?» deb so‘rashadi; (5) atamalar lug‘atini o‘rganing — ta’rif so‘raladigan savollarda atamalar uchraydi; (6) imtihondan oldin 6-bobdan barcha bug hunt-lar va 7-bobdan chizma (operator ustunligi, printf/scanf formatlari, bitta qatorga o‘n javob) takrorlang.

1.1. 4–7-haftalar va kurs fayllari

Har bir haftaga kurs papkasidagi fayl to‘plami mos keladi; ularning barchasi bu qo‘llanmada qayta ishlangan. Ma’ruzalar seshanba kunlari, laboratoriyalar darslik bo‘yicha (5-bob — 4-hafta, 6-bob — 5-hafta, 7-bob — 6-hafta, 8-bob — 7-hafta) o‘tiladi.

Hafta	Mavzu	Ma’ruzalar/laboratoriya	Qo‘shimcha fayllar
4	Ma’lumot turlari va operatorlar (Data Types and Operators)	$lecture_4$.txt — FoCS Ma’ruzasi 4 (2026-yil 6 oktyabr): takrorlash, o‘zgaruvchilar va ma’lumot turlari, operatorlar epilog · $Week_4$.txt — «Programming in C — Data Types and Operators» laboratoriyasi (darslikning 5-bobi)	—
5	Boshqaruv tuzilmalari (Control Structures)	$Lecture_5$.txt — Ma’ruzasi 5 (2026-yil 13 oktyabr): bloklar, shartlar, sikllar, funksiyalar, ko‘rish sohalari · Week-5-Control-Structures.txt — «Chapter 6 · Control structures» laboratoriyasi	bug_{huntc}.txt (1–4-masalalar: shartli tuzilmalar) · $bug_{huntloops}$.txt

Hafta	Mavzu	Ma’ruzalar/laboratoriya	Qo‘shimcha fayllar
6	Funksiyalar va strukturalar (Functions and Structures)	Lecture$_{6}$.txt — TashTech Ma’ruzasi 6 (2026-yil 20 oktyabr): boshqarish oqimi, I/O, satrlar, fayllar, buyruq qatori · Week-6-Functions-and-Structures.txt — «WEEK 6 · CHAPTER 7» laboratoriyasi: Scratch bloklaridan kofe mashinasigacha	bug$_{huntc}$.txt (5–6-masalalar: funksiyalar) · «kofe avtomati» loyihi (Stock, Recipe, Cup)
7	Ko‘rsatkichlar, satrlar va rekursiya (Pointers, Strings, Recursion)	Lecture$_{71}$.txt — TashTech Ma’ruzasi 7 (2026-yil 27 oktyabr): ko‘rsatkichlar va manzillar, massivlar va manzil arifmetikasi, satrlar, qidiruv va tartiblash · Recursion-Lecture.txt — «WEEK 7 · CHAPTER 8: Recursion»	bug$_{huntpointersstrings}$.txt · bug$_{huntpointersstringsanswerkey}$.txt (o‘qituvchi kaliti)

4–7-haftalar: mavzular va kurs fayllari

2. 4-hafta — Ma’lumot turlari va operatorlar (Data Types and Operators)

Hafta fayllari: Week$_{4}$.txt («Programming in C — Data Types and Operators» laboratoriyasi, 5-bob) · lecture$_{4}$.txt (FoCS Ma’ruzasi 4 — 2026-yil 6 oktyabr)

4-hafta — kursning asosi: C tilidagi o‘zgaruvchilar va ma’lumot turlari, ularning o‘lchamlari va aniqligi, butun va haqiqiy bo‘linish, arifmetik, mantiqiy, bitli operatorlar, inkrement/dekrement, ixcham tayinlash, ternar operator, tur o‘zgartirish va operatorlar ustunligi. Haftalaboratoriyasi: shar aniqligi (sphere$_{precision}$), kabisa yili qoidasini tuzatish (leap$_{year}$), xotiraga ko‘prik (baytlar va belgilar massivlari) va alifbo bo‘yicha birlashtirish (alphabetical$_{join}$); alohida LMS sessiyasida — Circle precision va Character processor vazifalari, besh harfli birlashtirish esa challenge. Quyida — barcha qoidalar, barcha savollar, testlar va hafta atamalari.

2.1. Asosiy g‘oyalar (4-haftaning "barcha qoidalari")

C ma’lumot turlari: e’lon qilish, o‘zgaruvchi nomlari, doimiy qiymatlar

C — tez, kichik, universal, platformadan mustaqil til; tizimli dasturlash uchun ishlatiladi (kompilyatorlar va interpretatorlar, operatsion tizimlar, ma’lumotlar bazalari, mikrokontrollerlar). C — statik (kompilyatsiya qilinadigan), turli, tuzilgan va buyruq beruvchi. "C is quirky, flawed, and an enormous success" — Ritchie. **Ob’ektning ma’lumot turi** xotirada qabul qilishi mumkin bo‘lgan qiymatlar to‘plamini va unga qo‘llanilishi mumkin bo‘lgan amallarni belgilaydi; C — zaif turli tilga ega: noaniq o‘zgartirishlarga va majburiy (potensial xavfli) tur o‘tkazishga (casting) ruxsat beradi.

C tur oilasi kichik: sonli (int, float, double), belgili (char), foydalanuvchi tomonidan belgilanadigan (struct, union). Asosiy laboratoriya jadvali:

Tur	C e’loni	Odatiy foydalanish
int	int score = 72;	butun sonlar
char	char grade = 'B';	bitta asosiy belgi
float	float temperature = 23.5f;	taxminiy haqiqiy qiymatlar
double	double radius = 3.0;	taxminiy haqiqiy qiymatlar
bool	bool passed = true;	true yoki false

Tur	C e’loni	Odatiy foydalanish
string	string name = "Ada";	matn (CS50 orqali)

O‘zgaruvchilar va turlar [Manba: Week$_4$, slayd 2]

Slayd tushuntirishlari: C11 da bool — <stdbool.h> dan makro; string — char * uchun CS50-typef (matnni emas, manzilni saqlaydi); bitta char — bitta bayt, "B" kabi "asosiy belgi", istalgan Unicode belgisi emas; void turidagi o‘zgaruvchi e’lon qilib bo‘lmaydi. **int main(void)**: int — qaytariladigan qiymat turi, qavs ichidagi void — "parametrsiz"; **void example(void)**: birinchi void — "qaytariladigan qiymat yo‘q", ikkinchisi — "parametrsiz".

Umumiy e’lon formati: **type variable-name [= value]**. Dars misollari:

```
char x;                /* uninitialized */
char x = 'A';          /* initialized to 'A' */
char x = 'A', y = 'B'; /* multiple variables initialized */
char x = y = 'Z';      /* multiple initializations */
```

[Manba: lecture$_4$, slayd 12]

O‘zgaruvchi nomlari qoidalari: nomlar harflar, raqamlar va _ ni o‘z ichiga olishi mumkin; harf bilan boshlanishi kerak; kalit so‘zlar (for, while va boshqalar) ishlatib bo‘lmaydi; nomlar katta-kichik harflarga sezgir — int x; va int X; ikki xil o‘zgaruvchini e’lon qiladi. Dars pop quiz: int money$owed; — noto‘g‘ri ($ belgisi); int total$_{count}$; — to‘g‘ri; int score2; — to‘g‘ri; int 2ndscore; — noto‘g‘ri (raqam bilan boshlanadi); int long; — noto‘g‘ri (kalit so‘z).

Doimiy qiymatlar — literal/fiksirlangan qiymatlar. Butun: int i = 3; long l = 3; unsigned long ul = 3UL; o‘nlik int i = 0xA; sakkizlik int i = 012;. Haqiqiy: float pi = 3.14159; float pi = 3.141F; double pi = 3.14159265358979323 84L;. Belgili: 'A', '\x41' (onlik), '\0101' (sakkizlik). Satrli: "hello world"; "hello" "world" — "hello world" bilan bir xil. Enumeratsiyalar: enum BOOL {NO, YES} → NO = 0, YES = 1; enum COLOR {R = 1, G, B, Y = 10} → G = 2, B = 3 — aniq qiymatsiz enumerator oldingisidan raqamlashni davom ettiradi.

[Manba: lecture$_4$, slaydlar 10–11; Week$_4$, slayd 2]

O‘lchamlar, diapazonlar va aniqlik: sizeof, float va double

Alohida o‘lchamlar mashina/kompilyatorga bog‘liq, lekin kafolatlangan: sizeof(char) < sizeof(short) <= sizeof(int) <= sizeof(long) va sizeof(char) < sizeof(short) <= sizeof(float) <= sizeof(double). Belgisiz versiyaning diapazoni belgilangan versiyaga qaraganda taxminan ikki barobar katta; belgilangan va belgisiz char faqat arifmetik ifodalarda farq qiladi. Ko‘p baytli sonlar uchun bayt tartibi muhim: **big endian** — yuqori bitlar (MSBs) pastki manzilda (powerpc protsessorlari; tarmoq tartibi — network order); **little endian** — pastki bitlar (LSBs) pastki manzilda (barcha x86-mos protsessorlar). Bu "NUXI muammosi".

Tur	signed	unsigned
short	short int x; short y;	unsigned short x; unsigned short int y;
standart holatda	int x;	unsigned int x;
long	long x;	unsigned long x;
float	float x;	N/A
double	double x;	N/A
char	char x; signed char x;	unsigned char x;

Sonli turlar: signed va unsigned [Manba: lecture$_4$, 7-slaid]

Laboratoriya jadvallari (macOS arm64 da o‘lchangan, 1 bayt 8 bit; boshqa platformalarda qiymatlar farq qilishi mumkin):

Tur	Bit	Diapazon
char	8	–128 … 127
signed char	8	–128 … 127
unsigned char	8	0 … 255
short	16	–32 768 … 32 767
unsigned short	16	0 … 65 535
int	32	–2 147 483 648 … 2 147 483 647
unsigned int	32	0 … 4 294 967 295
long	64	–9 223 372 036 854 775 808 … 9 223 372 036 854 775 807
unsigned long	64	0 … 18 446 744 073 709 551 615
long long	64	–9 223 372 036 854 775 808 … 9 223 372 036 854 775 807
unsigned long long	64	0 … 18 446 744 073 709 551 615

Butun sonli turlar: o‘lchamlar va diapazonlar; signed int = int, short int = short, long int = long [Manba: $Week_4$, 27–28-slaidlar]

Tur	Bit	O‘nlik aniqlik
float	32	6 ta muhim raqam
double	64	15 ta muhim raqam
long double	64	15 muhim raqam (bu platformadagi double kabi)

Haqiqiy turlar: FLT_{DIG}, DBL_{DIG}, $LDBL_{DIG}$ aniqlikdan (<float.h>) [Manba: $Week_4$, 29-slaid]

Laboratoriya xulosasi: **float** uchun 6, **double** uchun 15 muhim o‘nlik raqam (muhim raqamlar, verguldan keyingi belgilar emas); shar radiusi va hajmi uchun **double** dan foydalanamiz. Shar masalasi:

$$V = (4 / 3) \times \pi \times r^3$$

```
const double pi = 3.1415;
double radius = get_double("Radius: ");
// Today's input range: 0 < radius <= 1000
```

[Manba: $Week_4$, 4-slaid]

Bo‘lish va qoldiq: haftaning asosiy tuzuklari

Dars qoidasi: **bo‘lish amali operand turlari, emas, qabullovchi turiga qarab qanday ishlashini belgilaydi**. float x = 3/2; x = 1 (butun sonli bo‘lish); float x = 3.0/2; x = 1.5 (haqiqiy sonli bo‘lish); int x = 3.0/2; yana 1 (int ga aylantirish). Qoldiq: int x = 3%2; → 1; int y = 7; int x = y%4; → 3; int x = y%10; → 7.

Bo‘lish **tayinlashdan oldin** amalga oshiriladi — laboratoriya taxminlar jadvali:

Ifoda	Double saqlangan	%f bilan chop etilgan
4 / 3	1.0	1.000000
4.0 / 3.0	≈ 1.333333…	1.333333
(double)(4 / 3)	1.0	1.000000

Operand turlari / qanday ishlashini belgilaydi [Manba: $Week_4$, 6-slaid]

```
double fraction = 4 / 3;
printf("%f\n", fraction);
```

Laboratoriya taxmin 1: bu fragment **1.000000** ni chop etadi (A variant) — butun sonli bo‘lish 1 beradi, va faqat keyin tayinlash 1 ni double ga aylantiradi. Qabullovchi turining 4/3 ni bajarishiga ta’sir qiladimi? Yo‘q.

Formatlash saqlangan qiymatni yaxshirmaydi: radius = 3 bo‘lganda bir xil to‘g‘ri hajm **113.1** (%.1f) va **113.0940** (%.4f) sifatida chop etiladi — printf faqat ifodasini o‘zgartiradi, saqlangan sonni emas. Ruxsat etilgan radius: 0 dan katta va 1000 dan katta emas; nol, manfiy qiymatlar va 1000 dan katta barcha qiymatlar dastur tomonidan rad etiladi (musbat diapazon tekshiruvi sonli kiritishni ham kesib tashlaydi).

[Manba: Week$_4$, 3–8-slaidlar; lecture$_4$, 15-slaid]

Solishtirish va mantiqiy operatorlar: qisqa zanjir

Solishtirish operatorlari ikki operandni solishtirib "buliy" natija beradi: C da **har qanday noldan farqli qiymat (konvensiyaga ko‘ra 1) — to‘g‘ri, 0 — noto‘g‘ri**. Dars misollari: 3 > 2 → 1; 2.99 > 3 → 0; 3 >= 3 → 1; 3 < 3 → 0; 'A' < 'B' → 1; 3 == 3 → 1; 'A' == 'a' → 0; 3 != 3 → 0; 2.99 != 3 → 1. Tuzuklar: **== (tenglik)** — **= (tayinlash) emas**; haqiqiy o‘zgaruvchilarni solishtirish chekli aniqlik tufayli ishonchli emas.

Mantiqiy operatorlar && (VA), || (YOKI), ! (EMAS):

```
((9/3) == 3) && (2*3 == 6); /* evaluates to 1 */
('A' == 'a') && (3 == 3)    /* evaluates to 0 */
2 == 3 || 'A' == 'A';       /* evaluates to 1 */
2.99 >= 3 || 0              /* evaluates to 0 */
!(3 == 3);                  /* evaluates to 0 */
!(2.99 >= 3)                /* evaluates to 1 */
```

Qisqa zanjir (short circuit): shart qiymati allaqachon belgilanganda ifoda hisoblanishini to‘xtatadi; yon effektlarga e’tibor bering. Dars misollari: (3 == 3) || ((c = getchar()) == 'y') — ikkinchi qism HISOBLANMAYDI; (0) && ((x = x + 1) > 0) — ikkinchi qism HISOBLANMAYDI, x o‘zgarmaydi. Laboratoriya jadvali (belgiga kirishdan oldin massiv chegarasi):

i	i < 3	Belgilarni o‘qish?
0, 1, 2	true	ha
3	false	yo‘q

i < LENGTH && first[i] == second[i]: && chap noto‘g‘ri bo‘lganda o‘ng qismni o‘tkazib yuboradi [Manba: Week$_4$, 22-slaid]

[Manba: lecture$_4$, 16–18-slaidlar; Week$_4$, 22-slaid]

O‘sish/kamayish, ixcham tayinlash va ternar operator

Postfix: x++ — x = x+1 qisqa yozuvi; y = x++ x = x; x = x+1; ga teng — x **inkrementdan oldin** hisoblanadi (x-- ga o‘xshash). **Prefix**: ++x; y = ++x x = x+1; y = x; ga teng — x **inkrementdan keyin** hisoblanadi. Ixcham tayinlash operatorlari var = var (op) expr naqsh uchun:

```
x += 1   /* is the same as x = x+1 */
x -= 1   /* is the same as x = x-1 */
x *= 10  /* is the same as x = x*10 */
x /= 2   /* is the same as x = x/2 */
x %= 2   /* is the same as x = x%2 */
```

Ternar operator ?: — if (cond) x = <expra>; else x = <exprb>; naqsh uchun sintaktik shakar:

```
sign = x > 0 ? 1 : -1;
/* if (x > 0)
       sign = 1;
   else
       sign = -1; */
```

```
isodd = x % 2 == 1 ? 1 : 0;
/* if (x % 2 == 1)
       isodd = 1;
   else
       isodd = 0; */
```

[Manba: lecture$_4$, 19–23-slaidlar]

Bitli operatorlar

Operator	Ma’no	Misollar
&	AND	0x77 & 0x3 → 0x3; 0x77 & 0x0 → 0
|	OR	0x700 | 0x33 → 0x733; 0x070 | 0 → 0x070
^	XOR	0x770 ^ 0x773 → 0x3; 0x33 ^ 0x33 → 0
<<	chapga siljish	0x01 << 4 → 0x10; 1 << 2 → 4
>>	o‘ngga siljish	0x010 >> 4 → 0x01; 4 >> 1 → 2

Bitli operatorlar [Manba: lecture$_4$, 21-slaid]

Slaid qoidalari: AND ikkala operand ham to‘g‘ri bo‘lganda to‘g‘ri; OR — kamida bittasi to‘g‘ri bo‘lganda; XOR — aynan bittasi to‘g‘ri bo‘lganda.

Tur o‘zgartirish va operator ustunligi

Yuqori aniqlikka ko‘tarilganda ma’lumotlar saqlanadi — kompilyator aralash turdagi ifodalarda buni avtomat ravishda amalga oshiradi; qoida (istisnolar bilan): ikkilik ifodaning har bir a’zasi eng yuqori aniqlikdagi operand aniqligiga ko‘tariladi. char → int avtomatik o‘zgarishi belgilarni solishtirish va "hisoblash"ga imkon beradi:

```
int i;
float f;
f = i + 3.14159; /* i is promoted to float,
                    f = (float)i + 3.14159 */

isupper = (c >= 'A' && c <= 'Z') ? 1 : 0;
/* c and literal constants are converted to int */
if (!isupper)
    c = c - 'a' + 'A';
/* subtraction is possible because of integer conversion */
```

Ustunlik va hisoblash tartibi: ++, −−, (cast), sizeof — eng yuqori ustunlik; *, /, % +, – dan yuqori; ==, != &&, || dan yuqori; tayinlash operatorlari — juda past. Ikki ma’nolilik va ustunlik yon effektlaridan qochish uchun qavs ( ) ni ko‘p ishlating:

```
y = x*3+2            /* same as y = (x*3)+2 */
x != 0 && y == 0     /* same as (x != 0) && (y == 0) */
d = c >= '0' && c <= '9'  /* same as d = (c >= '0') && (c <= '9') */
```

[Manba: lecture$_4$, 24–25-slaidlar]

Laboratoriya: kabisa yili va belgilar massivi

Kabisa yili. Oddiy yil — 365 kun (fevralda 28), kabisa yili — 366 (29 fevral): qo‘shimcha kun taqvimni mavsumlar bilan mos kelishiga xizmat qiladi. Qoida yuqoridan pastga o‘qiladi: 400 ga karrali? → kabisa; aks holda 100 ga karrali? → oddiy; aks holda 4 ga karrali? → kabisa; aks holda oddiy. 2000 — kabisa (400 ga karrali); 1900 — oddiy (100 ga karrali, lekin 400 ga emas); 2024 — kabisa; 2025 — oddiy; 2100 — oddiy (chegira tekshiruvi: 100 ga karrali, lekin 400 ga emas). Laboratoriyaning ixcham ifodasi:

```
year % 400 == 0 || (year % 4 == 0 && year % 100 != 0)
```

Takroriy kirish ("Go back to line 1" — CS50 uslubidagi takrorlanishning psevdokodi): sikldan oldin yilni o‘qing va sikl oxirida yana; yil 0 yoki manfiy bo‘lsa, klassifikatsizsiz tugatadi; 9999 dan katta — xato xabari va to‘xtatish. Agar takroriy o‘qishni unutsangiz, musbat yil hech qachon o‘zgarmaydi, sikl abadiy takrorlanadi. Birinchi qoida year % 4 == 0 2024 va 2025 ni to‘g‘ri klassifikatsiyalaydi — bu ikki test asr xatolarini ochib bermaydi; 1900 va 2000 ni tekshiring.

Xotlik ko‘prik. RAM moduli → bir chip → konseptual panjara: har bir kvadrat — bir bayt; sizeof(char) — C da 1 bayt, bitlar soni CHAR_{BIT} ni beradi (o‘quv mashinasida 8); o‘lchangan int hajmi — 4 bayt. Uchta belgidan iborat massivlar:

```
char first[3]  = {'d', 'o', 'g'};
char second[3] = {'c', 'a', 't'};
```

Indeks	0	1	2
first	d	o	g
second	c	a	t

Indekslar 0–2 · terminator '\0' yo‘q · bu C-stringlar emas [Manba: Week_4, 18-slaid]

Alifbo bo‘yicha yopishtirish (dog + cat → catdog): ikki massivni birinchi farq qiluvchi belgiga butunlay solishtiramiz (car/cat indeks 2 da farq qiladi: r va t), so‘ngra "kichikroq" massivni birinchi olib ketamiz. Teng prefiks skani: i < LENGTH && first[i] == second[i] — chegaraga belgiga murojaat qilishdan oldin bo‘lishi kerak; teng massivlar chegara yetib boradi: $\text{first}_{\text{earlier}}$ = i == LENGTH || first[i] < second[i]. Nusxa ko‘chirish: i joined[0..2] ni to‘ldiradi, LENGTH + i — joined[3..5]:

Index	0	1	2	3	4	5
joined	c	a	t	d	o	g

Nusxa ko‘chirish siljishi: i 0–2 ni to‘ldiradi, LENGTH + i 3–5 ni to‘ldiradi [Manba: Week_4, 23-slaid]

Chop etish: da yakuniy '\0' yo‘q, shuning uchun %s mos kelmaydi — aynan oltita belgini **%c** orqali chop etamiz.

[Manba: Week_4, 9–26-slaidlar]

2.2. 4-hafta savollari — ma’ruza va laboratoriya savollari model javoblar bilan

S1. Laboratoriya isingiz: butun sonlarni, o‘nli kasr qiymatlarini va bitta belgini saqlaydigan turilar qaysilar?

Javob. int — butun sonlar uchun; float yoki double — o‘nli (haqiqiy) qiymatlar uchun; char — bitta belgi uchun. Ijro main dan boshlanadi; stdio.h printf ni, cs50.h esa $\text{get}_{\text{double}}$ ni e’lon qiladi.

S2. int main(void) va void example(void) dagi void nima degani?

Javob. int main(void) da int — qaytariladigan qiymat turi, qavs dagi esa "parametrlar yo‘q" degani. void example(void) da birinchi void "qaytariladigan qiymat yo‘q", ikkinchisi esa "parametrlar yo‘q" degani. Odatdagi void turidagi o‘zgaruvchini e’lon qilib bo‘lmaydi.

S3. Shar va radiusi uchun double emas float tanlangan?

Javob. Radius kasr qismga ega bo‘lishi mumkin, va platformamizda double floatdan aniqroq: 15 ta ahamiyatli o‘nli raqamlar 6 taga nisbatan (DBL_{DIG} va FLT_{DIG}). Biroq double pi = 3.1415 ni aniq qilmaydi — haqiqiy qiymatlar taxminan bo‘lib qoladi.

S4. double fraction = 4 / 3; printf("%f\n", fraction); nima chiqaradi?

Javob. 1.000000. Bo‘lish tayinlanishgacha amalga oshiriladi: ikkala operand ham butun son, 4/3 1 beradi va faqatgina keyin 1 double ga aylanadi. Qabul qiluvchi tur bo‘lishni qanday amalga oshirishiga ta’sir qilmaydi — operandlar turlari hal qiluvchi rol o‘ynaydi. Xuddi shunday (double)(4/3) 1.0 beradi, 4.0/3.0 esa 1.333333.

S5. Shfera dasturi radius = 3 bo‘lganda %.1f va %.4f orqali bir xil hajmi chiqaradi. ortiqcha belgilar saqlangan qiymatni yaxshilaydimi?

Javob. Yo‘q. Chiqarishlar — 113.1 va 113.0940: bu bir xil saqlangan son; formatlash faqat ko‘rsatishni o‘zgartiradi (printf aniqlikni ko‘rsatishni boshqaradi, saqlangan qiymatni emas).

S6. Darsning Pop quiz II: int x = 017; int y = 12; — x y dan katta?

Javob. Ha. 017 — sakkizlik konstanta (jadvaldagi int i = 012; kabi — sakkizlik son): $017_8 = 1 \cdot 8 + 7 = 15$, va $15 > 12$.

S7. enum sz {S = 0, L = 3, XL}; va enum sz {S = 0, L = −3, XL}; da XL qanday qiymatga ega?

Javob. Aniq qiymati bo‘lmagan enumerator avvalgisidan keyin raqamlashni davom ettiradi — enum COLOR {R = 1, G, B, Y = 10} kabi, bu yerda G = 2, B = 3. Shuning uchun birinchi holatda XL = 4, ikkinchisida XL = −2.

S8. Nima uchun 'A' < 'B' 1 beradi, 'A' == 'a' esa 0 beradi?

Javob. char avtomatik ravishda int ga (belgining kodi) aylanadi. 'A' va 'B' kodi o‘sish tartibida keladi, shuning uchun kodlarni solishtirish 1 beradi; 'A' va 'a' kodlari har xil, shuning uchun tenglik noto‘g‘ri. Xuddi shu o‘zgarish tufayli c = c - 'a' + 'A'; (kichik harfni katta harfga aylantirish) arifmetikasi mumkin.

S9. Qisqa zanjir nima va (0) && ((x = x + 1) > 0) da x bilan nima bo‘ladi?

Javob. Ifoda hisoblanishi to‘xtatiladi, agar natija allaqachon aniqlangan bo‘lsa. Chap operand && 0 ga teng, demak butun ifoda noto‘g‘ri va o‘ng qismi — yon ta’sirli x = x + 1 — hech qachon hisoblanmaydi: x o‘zgarmaydi. Xuddi shu (3 == 3) || ((c = getchar()) == 'y') da getchar chaqirilmaydi. i < LENGTH && first[i] == second[i] tsiklida i = 3 bo‘lganda first[3] ga murojaat qilinmaydi.

S10. y = x++ va y = ++x o‘rtasidagi farq nima?

Javob. Postfiks: y = x++; bu y = x; x = x + 1; ga teng — x inkrementdan oldin hisoblanadi. Prefiks: y = ++x; bu x = x + 1; y = x; ga teng — x inkrementdan keyin hisoblanadi. Dekrement uchun ham shunday: y = x−− va y = −−x.

S11. sign = x > 0 ? 1 : −1; ifodasida uchlik operatori qanday ishlaydi?

Javob. Bu if (x > 0) sign = 1; else sign = −1; uchun sintaktik shakar: shart ? dan oldin, to‘g‘ri bo‘lgandagi qiymat ? va : orasida, noto‘g‘ri bo‘lgandagi qiymat : dan keyin. Kod qisqaroq va tushunarliroq bo‘ladi; slaydning ikkinchi misoli — isodd = x % 2 == 1 ? 1 : 0;.

S12. Sharh hajmini hisoblovchi dastur qays kirish ma’lumotlarini rad etishi kerak?

Javob. Nol, manfiy radiuslar va 1000 dan katta qiymatlar: ruxsat berilgan diapazon 0 < radius <= 1000. Musbat diapazonni tekshirish shuningdek, son bo‘lmagan (nonfinite) kirishni rad etadi.

S13. Nima uchun 1900 oddiy yil, 2000 esa kabisa yili? Nima uchun 2100 oddiy?

Javob. 2000 400 ga karrali — kabisa yili. 1900 100 ga karrali, lekin 400 ga emas — oddiy yil; 2100 ham shunday (100 ga karrali, lekin 400 ga emas), qoida 100 ga karralilikni tekshirishdan oldin 400 ga karralilikni tekshiradi. Qisqa shart: year % 400 == 0 || (year % 4 == 0 && year % 100 != 0); 2024 uchun bu kabisa yil, 2025 oddiy.

S14. Nima uchun char first[3] = {'d', 'o', 'g'} massivini %s bilan chop etib bo‘lmaydi va bu && tartibiga qanday bog‘liq?

Javob. Massivda aynan uchta belgi va '\0' terminatori yo‘q — bu C-stroka emas: %s massivdan keyin xotirani tasodifiy nol baytgacha o‘qishda davom etardi. Uchta belgini %c orqali aynan chop etiladi. Solishtirish tsiklida chegarasi birinchi bo‘lishi shart: i < LENGTH && first[i] == second[i] — i = 3 bo‘lganda chap qismi noto‘g‘ri, && o‘ng qismini o‘tkazib yuboradi va massiv chegarasidan chiqib ketish holati yuz bermaydi.

2.3. 4-haftaviy testlar

T1. C ning qaysi turi bitta asosiy belgini saqlaydi?

a) int
b) char
c) double
d) bool

Javob: b) char

Tushuntirish: char grade = 'B'; — bitta asosiy belgi; int — butun sonlar, double — taxminiy haqiqiy sonlar, bool — true/false.

T2. Kurs platformasida double da nechta ahamiyatli o‘nlik raqam bor?

a) 6
b) 8
c) 15
d) 32

Javob: c) 15

Tushuntirish: DBL_{DIG} = 15 (float da — 6, FLT_{DIG}). Bu ahamiyatli raqamlar, verguldan keyingi raqamlar soni emas; long double bu platformada ham 15.

T3. double fraction = 4 / 3; printf("%f\n", fraction); nimani chiqaradi?

a) 1.000000
b) 1.333333
c) kompilyatsiya xatosi
d) 0.000000

Javob: a) 1.000000

Tushuntirish: 4/3 — butun sonli bo‘lish, 1 beradi; double ga o‘tkazish tayinlanganda amalga oshiriladi. To‘g‘ri javob laboratoriya 1 da — A variant.

T4. Qaysi yozuv butun son 15 ni bildiradi?

a) int x = 0xA;
b) int x = 017;
c) int x = 15.0;
d) int x = '15';

Javob: b) int x = 017;

Tushuntirish: 017 — sakkizlik konstanta: 1·8 + 7 = 15. 0xA — o‘nliklik 10; '15' — belgi (bitta tirnoq ichida faqat bitta belgi bo‘ladi).

T5. Qaysi o‘zgaruvchi e’loni to‘g‘ri?

a) int money$owed;

b) int 2ndscore;

c) int total_count;

d) int long;

Javob: c) int total_count;

Tushuntirish: ma'ruza testi: $ ruxsat etilmaydi; nom raqam bilan boshlanmaydi; long — kalit so'z. Harflar, raqamlar va _ — ruxsat etiladi, lekin birinchisi harf bo'lishi kerak.

T6. int y = 7; bo'lganda y % 10 qiymati qancha?

a) 0

b) 3

c) 7

d) 1

Javob: c) 7

Tushuntirish: int y = 7; int x = y % 10; / natija 7 / — 7 10 dan kichik, qoldiq o'ziga teng; y % 4 esa 3 beradi.

T7. 0x01 << 4 nimaga teng?

a) 0x10

b) 0x01

c) 4

d) 0x14

Javob: a) 0x10

Tushuntirish: chapga 4 bitga siljish: 0x01 << 4 ifodasi 0x10 ni beradi; xuddi shunday 1 << 2 → 4, 0x010 >> 4 → 0x01, 4 >> 1 → 2.

T8. Qaysi operatorlar guruhi eng yuqori ustunlikka ega?

a) = va +=

b) && va ||

c) * va /

d) ++, −−, (cast), sizeof

Javob: d) ++, −−, (cast), sizeof

*Tushuntirish: ++, −−, (cast), sizeof — eng yuqori ustunlik; keyin *, /, %; keyin +, −; ==, != &&, || dan yuqori; tayinlash — juda past ustunlik.*

T9. y = x*3 + 2 ifodasi ... ga teng

a) y = x*(3+2)

b) y = (x*3)+2

c) y = x+(3*2)

d) y = 3*(x+2)

Javob: b) y = (x*3)+2

Tushuntirish: , /, % belgilari + va − belgilariga nisbatan ustunlikka ega, shuning uchun avval ko'paytirish amalga oshiriladi — slayddagi "same as y = (x3)+2" misolida kabi.

T10. Ifodada (0) && ((x = x + 1) > 0) bo'lganda x bilan nima bo'ladi?

a) x 1 ga ortadi

b) o'zgarmaydi

c) kompilyatsiya xatosi

d) x 0 bo'ladi

Javob: b) o‘zgarmaydi

Tushuntirish: qisqa tutashuv: chap operand && allaqachon noto‘g‘ri, shuning uchun yon effektlari bo‘lgan o‘ng qism hisoblanmaydi — "be careful of any side effects".

2.4. Atamalar — 4-hafta glossariysi

Atama	Ma’nosi
Ma’lumot turi (datatype)	ob’yektning xotiradagi qiymatlari to‘plamini va unga ruxsat etilgan amallarni belgilaydi; C — avtomatik konvertatsiya va cast (o‘tkazish) bilan zaif tipizatsiyalangan til
O‘zgaruvchi (variable)	xotirada saqlanadigan qiymatga yoki hisoblanadigan ifodaga nomlangan bog‘lanish/siljish (int x = 0, y = 0; y = x + 2;)
Ifoda (expression)	qiymatlar, o‘zgaruvchilar, operatorlar va funksiyalarning kombinatsiyasi (y = x + 2)
Operator (operator)	ob’yekt ustida manipulyatsiyani belgilaydi; birlikli (–, ++), ikkilik (+, –, *, /), uchlik (?:)
Butun sonli bo‘lish	ikki butun sonning bo‘linishi: kasr qismi tashlanadi (4/3 → 1, 3/2 → 1); qabullovchi turi ta’sir qilmaydi — operand turlari muhim
Qoldiq (modulus, %)	butun sonlarning bo‘linishidan qoldiq: 3 % 2 → 1, 7 % 4 → 3, 7 % 10 → 7
Ahamiyatli raqamlar	haqiqiy turdagi aniqlik: float — 6, double — 15 ahamiyatli o‘nli raqamlar (<float.h> dan FLT_{DIG}, DBL_{DIG})
Bayt tartibi (endianness)	big endian — yuqori baytlar pastki manzilda (powerpc, network order); little endian — pastki baytlar pastki manzilda (x86); "NUXI muammosi"
Konstanta (constant)	literal/fixlangan qiymat: 3, 3UL, 0xA (hex), 012 (octal), 'A', '\x41', "hello world"
Enumeratsiya (enum)	nomlangan butun sonli konstantalar; aniq qiymat bo‘lmasa, raqamlash davom etadi: enum COLOR {R=1,G,B,Y=10} → G=2, B=3
Qisqa tutashuv (short circuit)	&& va || hisoblashni to‘xtatish, natija allaqachon aniqlanganda; o‘ng qismning yon effektlari bajarilmasligi mumkin
Operator ustunligi (precedence)	tartib: ++/−−/(cast)/sizeof → *,/,% → +,– → ==,!= → &&,|| → tayinlash; qavs ( ) — ko‘p foydalaning
Increment/decrement	x++ (postfiks: orttirishdan oldingi qiymat) va ++x (prefiks: orttirishdan keyingi qiymat)
Kabisa yili (leap year)	400 ga bo‘linadigan yoki 4 ga bo‘linib 100 ga bo‘linmaydigan yil: year % 400 == 0 || (year % 4 == 0 && year % 100 != 0)

3. 5-hafta — Boshqaruv konstruksiyalari (Control Structures)

Hafta fayllari: Week-5-Control-Structures.txt (laboratoriya "Chapter 6 · Control structures") · Lecture_5.txt (Dars 5 — October 13, 2026)

5-hafta dasturni "qaror qabul qilish va vazifalarni takrorlash"ni o‘rgatadi: bloklar va murakkab operatorlar, shartlar (if / else if / else, switch), tsikllar (while, for, do-while), break va continue, va darsda — funksiyalar, modulli dasturlash va ko‘rinish maydonlari (static, register, extern). Laboratoriya kitobning 6-bobida o‘tadi: Melody misoli (ismdan melodiyani), CS50 uslubida satr uzunligini hisoblash, ixtiyoriy do-while va continue fragmentlari. Quyida — haftaning barcha qoidalar, savollar, testlar va terminlari.

3.1. Asosiy tushunchalar («5-hafta barcha qoidalar»)

Bloklar va murakkab operatorlar

Oddiy operator nuqta-vergul bilan tugaydi: z = foo(x+y);. Bir nechta operatorlar qavsli qavslar {} bilan murakkab operator (blok) birlashtiriladi. Blok oddiy operator kutilayotgan joyda turishi mumkin; bitta butun sifatida kompilyatsiya qilinadi; ichida o‘zgaruvchilar e’lon qilinishi mumkin; blok bo‘lishi bo‘sh {} bo‘lishi mumkin; yopuvchi qavsdan keyin nuqta-vergul qo‘yilmaydi. Bloklarni bir-biriga joylashtirish mumkin:

```
{
    int temp = x + y;
    z = foo(temp);
    {
        float temp2 = x * y;
        z += bar(temp2);
    }
}
```

[Manba: Lecture$_5$, slaydlar 4–6]

C va if / else if / else oilasidagi shart

C++ yoki Java dan farqli o‘laroq, C89/C90 da mantiqiy tur yo‘q; C99 da bool <stdbool.h> orqali mavjud. **Shart — bu ifoda (yoki ifodalar seriyasi)**, masalan n < 3 yoki x < y || z < y. Nol bo‘lmagan ifoda ⇒ shart to‘g‘ri; ifoda sonli (yoki ko‘rsatkichli) bo‘lishi kerak:

```
const char str[] = "some text";
if (str) /* string is not null */
    return 0;
```

if: shartni hisoblang; agar to‘g‘ri bo‘lsa — ichki operatorni bajar; aks holda — hech narsa qilmaydi. **else** (ixtiyoriy): agar shart noto‘g‘ri bo‘lsa, operatorni bajar. **else if**: qo‘shimcha alternativ yo‘llar; shartlar birinchi to‘g‘ri bo‘lguncha ketma-ket tekshiriladi — agar bir nechtasi to‘g‘ri bo‘lsa, faqat birinchasi bajariladi; ichma-ich if ga teng. Har qanday ichki operator blok bo‘lishi mumkin:

```
if (x % 2) y += x / 2;

if (x % 2 == 0)
    y += x/2;
else
    y += (x+1)/2;

if (x % 2 == 0) y += x / 2;
else if (x % 4 == 1) y += 2 * ((x + 3) / 4);
else
    y += (x + 1) / 2;
```

[Manba: Lecture$_5$, slaydlar 7–11]

Ichma-ich if — klassik tuzoq: qavsli bo‘lmagan else **yaqin ichki** if bilan bog‘lanadi. Else ni tashqi if bilan bog‘lash uchun — qavsli qavslar:

```
if (x % 4 == 0)
    if (x % 2 == 0) y = 2;
    else
        y = 1;
/* else ichki ifga tegishli! */

if (x % 4 == 0) {
    if (x % 2 == 0) y = 2;
} else
    y = 1;
```

```
/* endi else tashqi if bilan bog'langan */
```

[Manba: Lecture 5, slaydlar 12–13]

switch: case, break, default va fall-through

switch — alternativ shartli operator: kirishda butun sonli (yoki belgili) o‘zgaruvchi; uning qiymatlari uchun holatlar (case) ko‘rib chiqiladi. switch tarkibi — blok; **case-metkalari — blokka kirishning turli nuqtalari**; kompilyator o‘zgaruvchini har bir metka bilan ketma-ket solishtiradi; mos kelganda, u eng yaqin break gacha bo‘lgan ichki kodni bajaradi; break bo‘lmaydigan holda bajarish «pastga tushadi» (falls through) keyingi case ga yangi tekshirishsiz; default — «aks holda» tarmog‘i. C laboratoriya tamoyillari: switch butun sonli ifodani (shu jumladan char ni) qabul qiladi; case-metkalari — turli butun sonli doimiy ifodalar; diapazonlar va murakkab shartlar uchun if dan foydalaning. Laboratoriyaning to‘liq misoli (grade):

```
if (grade == 'A') {
    printf("Perfect!\n");
} else if (grade == 'B') {
    printf("You did good!\n");
} else if (grade == 'C') {
    printf("You did okay!\n");
} else if (grade == 'D') {
    printf("At least it's not an F!\n");
} else if (grade == 'F') {
    printf("YOU FAILED!\n");
} else {
    printf("That's not a valid grade!\n");
}
```

```
switch (grade) {
case 'A':
    printf("Perfect!\n");
    break;
case 'B':
    printf("You did good!\n");
    break;
case 'C':
    printf("You did okay!\n");
    break;
case 'D':
    printf("At least it's not an F!\n");
    break;
case 'F':
    printf("YOU FAILED!\n");
    break;
default:
    printf("That's not a valid grade!\n");
}
```

if/else-if zinasidagi har bir tenglik tekshiruvi case-metkasiga, esa else — default ga aylanadi; break switch dan chiqadi (bajarish yopuvchi qavsdan keyin davom etadi — break dasturni tugatmaydi). Ikkala dastur ham barcha A/B/C/D/F va W baholari uchun bir xil xabarlarni beradi. Bir nechta case-metkalari ketma-ket joylashtiriladi:

```
switch (ch) {
case 'Y':  /* ch == 'Y' */
    /* do something */
    break;
case 'N':  /* ch == 'N' */
    /* do something else */
    break;
default:   /* otherwise */
```

```
    /* do a third thing */
    break;
}

switch (ch) {
case 'Y':
case 'y':
    /* do something if ch == 'Y' or ch == 'y' */
    break;
}
```

[Manba: Week-5-Control-Structures, slaydlar 5–6; Lecture$_5$, slaydlar 14–15]

6.1-misol: ismdan melodiyalar

6.1-misol kitobi belgilarni birma-bir o‘qiydi, ASCII-kodni 7 ga bo‘lgandagi qoldiqni hisoblaydi va notaning nomini chiqaradi (tovar chiqarilmaydi). 'A' uchun: kod 65; 65 % 7 = 2; kutilgan nota — C. 01$_{\text{melodybuggy}}$.c ta’lim versiyasida case 2 dan **break etishmaydi** — bu tahlil uchun mo‘ljallato‘g‘ri xato: kirish A# chiqaradi C, keyin D (case 3 ga yangi tekshirishsiz pastga tushadi). debug50 da tahlil: printf case 2 da to‘xtash nuqtasi, case 3 da Step Over; tuzatish — case 2 da printf dan keyin break qo‘shish; B# va darhol # ni tekshirish. Tuzatilgan versiya belgini o‘qiydi, muvaffaqiyatli kirishni va qo‘riqchi belgini # (scanf(" %c", &C) bo‘shliq belgilarini o‘tkazib yuboradi) tekshiradi, notani tanlaydi va yana o‘qiydi; EOF va # tsiklni tugatadi; case ichidagi break ni tark etgani uchun yangi o‘qish amalga oshiriladi. Kirish AB# beradi C notani, keyin D notani.

Belgilar	ASCII-kod	65 % 7	Kutilgan nota
A	65	2	C

Melodiya: 'A' → kod 65 → qoldiq 2 → nota C [Manba: Week-5-Control-Structures, slayd 7]

Tsikllar: while, for, do-while

while — eng oddiy tsikl: shart to‘g‘ri bo‘lguncha jismni hisoblang. Shart **oldin** jismdan hisoblanadi, shuning uchun jism hech qachon bajarilmaydi:

```
while (/* condition */)
    /* loop body */
```

for — «sanauvchi» tsikl: qavsda nuqta-vergul bilan ajratilgan uchta ifoda — boshlash (i = 1), shart (i <= n), oshirish (i++). Ifodalar bo‘sh bo‘lishi mumkin (bo‘sh shart to‘g‘ri deb hisoblanadi). Bajarish tartibi: boshlash bir marta; jismdan oldin shart; jism; yangilanish; yana shart. While ga teng va vergul operatori bilan variant (vergul — pastroq ustunlikdagi operator, chapdan o‘ngga hisoblanadi; bu funksiya argumentlari orasidagi vergul bilan bir xil emas):

```
int factorial(int n) {
    int i, j = 1;
    for (i = 1; i <= n; i++) j *= i;
    return j;
}

int factorial(int n) {
    int j = 1;
    int i = 1; /* initialization */
    while (i <= n /* condition */) {
        j *= i;
        i++; /* increment */
    }
    return j;
}

int factorial(int n) {
```

```
    int i, j;
    for (i = 1, j = 1; i <= n; j *= i, i++)
        ;
    return j;
}
```

Laboratoriya taqqoslashi (§6.2.1): ikkala fragment ham **1 2 3** ni chiqaradi; while variantida int i = 1; sanagich tsikldan oldin e’lon qilinadi, for variantida esa sarlavhada va bu sanagich faqat tsikl ichida ko‘rinadi (undan keyin unga murojaat qilmang):

```
int i = 1;
while (i <= 3)
{
    printf("%d ", i);
    i++;
}

for (int i = 1; i <= 3; i++)
{
    printf("%d ", i);
}
```

do-while while dan shu jihatdan farq qiladiki, shart har bir iteratsiyadan **keyin** hisoblanadi: jism kamida bir marta bajariladi; oxiridagi nuqta-vergulga e’tibor bering:

```
char c;
do {
    /* loop body */
    puts("Keep going? (y/n) ");
    c = getchar();
    /* other processing */
} while (c == 'y' /* other conditions */);
```

[Manba: Lecture$_5$, slaydlar 18–22; Week-5-Control-Structures, slaydlar 10–11]

break va continue

Ba’zan tsiklni oldindam tugatish kerak bo‘ladi. **break;** eng ichki tsikldan yoki switch operatoridan chiqadi:

```
char c;
do {
    /* loop body */
    puts("Keep going? (y/n) ");
    c = getchar();
    if (c != 'y')
        break;
    /* other processing */
} while (/* other conditions */);
```

continue; eng ichki tsikl jismining qolgan qismini o‘tkazib yuborib, uning shartiga o‘tadi. Darsdagi misol — min makrosi bilan EKUB (gcd): agar i a ni bo‘lmasa, keyingi i ga o‘tamiz; agar i a va b ni bo‘lsa — eslab qolamiz:

```
#define min(a,b) ((a)<(b)?(a):(b))

int gcd(int a, int b) {
    int i, ret = 1, minval = min(a, b);
    for (i = 2; i <= minval; i++) {
        if (a % i) /* i not divisor of a */
            continue;
        if (b % i == 0) /* i is divisor of both a and b */
            ret = i;
    }
```

```
    return ret;
}
```

[Manba: Lecture$_5$, slaydlar 23–24]

Funksiyalar, ko‘lamilar va modullar (5-dars bo‘yicha)

5-dars laboratoriya chegarasidan chiqib va funksiyalar bilan tanishtiradi: **divide and conquer** — dasturni oddiyroq kichik masjalarga bo‘lish. Misol — chiziqli Diofant tenglamasi ax + by = c: a, b, c ni oling; g = gcd(a, b) ni hisoblang; agar c gcd ga karrali bo‘lmasa — yechim yo‘q; kengaytiril Evklid algoritmini ishga tushiring; x va y ni (c/g) ga masshtablang; yechimni chiqaring. Evklid algoritmi gcd(a, b) = gcd(b, a mod b) ga asoslanadi:

```
int gcd(int a, int b) {
    while (b) { /* if a < b, performs swap */
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}
```

Funksiyalar bittadan ortiq qiymat qaytarmaydi: bir nechta chiqishlar uchun (kengaytirilgan Evklid gcd, x va y ni qaytaradi) darsda global o‘zgaruvchilar — funksiyadan tashqari e’langan o‘zgaruvchilar ishlatiladi: ular dastur butun hayoti davomida mavjud, har qanday funksiyadan foydalanish mumkin. **Modul** C da: interfeys (sarlavha fayli .h) + amalga oshirish (yordamchi manba/obyekt fayllari). euclid.h sarlavhasi funksiya prototiplari va global o‘zgaruvchilar extern e’lonlarini o‘z ichiga oladi; extern kompilyatorga o‘zgaruvchi boshqa faylda aniqlanganligini bildiradi; ikkala faylni kompilyatsiya va linklash kerak: gcc -g -O0 -Wall diophant.c euclid.c -o diophant.o:

```
/* ensure included only once */
#ifndef EUCLID_H
#define EUCLID_H
/* global variables (declared in euclid.c) */
extern int x, y;
/* compute gcd */
int gcd(int a, int b);
/* compute g = gcd(a, b) and solve ax+by=g */
int ext_euclid(int a, int b);
#endif
```

Ko‘lam (scope) — o‘zgaruvchining amal qilish hududi; odatda bu o‘zgaruvchi e’lon qilingan blokdir. Funksiyadan tashqaridagi o‘zgaruvchilar — global. Dars misoli: global int nmax = 20; va main dagi Fibonachchi tsikli; agar main ichida lokal int nmax = 25; e’langan bo‘lsa, u **globalni yashiradi** — tsikl 25 gacha yetadi va 25 qator chiqariladi. **static**: funksiya tashqarisida — o‘zgaruvchi/funksiya faqat fayl ichida ko‘rinadi (extern bo‘lmaydi); funksiya ichida — o‘zgaruvchi lokal, lekin dastur ishga tushirilganda bir marta initilashtiriladi va chaqirishlarda qayta initilashtirilmaydi (static int somePersistentVar = 0;). **register**: o‘zgaruvchini registrda saqlashga aniq so‘rov; faqat lokal o‘zgaruvchilar va argumentlar; ortiqcha/ruhsiz e’lonlar e’tiborga olinmaydi; register-o‘zgaruvchining manzilini olish mumkin emas — unga ko‘rsatkich ruxsat etilmaydi.

```
int nmax = 20;
/* The main() function */
int main(int argc, char **argv) /* entry point */
{
    int a = 0, b = 1, c, n;
    printf("%3d: %d\n", 1, a);
    printf("%3d: %d\n", 2, b);
    for (n = 3; n <= nmax; n++) {
        c = a + b;
```

```
            a = b;
            b = c;
            printf("%3d: %d\n", n, c);
        }
        return 0; /* success */
    }
```

[Manba: Lecture$_5$, slaydlar 25–43]

Laboratoriya: satr uzunligi, do-while va continue

Satr uzunligi (CS50-qo‘shimcha): string name = get$_{string}$("Name: "); satr H, I, ! va terminatori '\0' saqlaydi. Algoritm: nol dan boshlab sanash; har bir belgini sanash; oxirgi belgida to‘xtash; **terminator hisoblanmaydi**. HI! uchun: n = 0 H ni o‘qiydi, n = 1 I ni o‘qiydi, n = 2 ! ni o‘qiydi, n = 3 NUL ga yetadi — keyingi oshirishgacha shart noto‘g‘ri, 3 chiqariladi. Bo‘sh satr 0 beradi. strlen va ko‘rsatkichsiz.

do-while (ixtiyoriy fragment §6.2.2, 05$_{textdowhile}$.c fayli): satir belgilarini, chiziq o‘tishdan tashqari, va **qo‘riqchi belgini %** ham saqlaydi — % kirganda shart noto‘g‘ri bo‘lishidan oldin bitta belgi saqlanadi; shart kirganni va sig‘imi (Counter < MaxLength) tekshiradi; matn — C-satri emas. **continue** (ixtiyoriy fragment §6.4, 06$_{skipletters}$.c fayli): chiziq o‘tishdan boshqa besh belgini o‘qiydi va harflarini o‘tkazib yuboradi; 'A' uchun printf ni o‘tkazib, i++ yangilanishiga o‘tadi; '1' uchun printf belgini ishlaydi, keyin xuddi shu yangilanish; A1b2! kirgisi 1, 2 va ! ni qoldiradi.

[Manba: Week-5-Control-Structures, slaydlar 12–14]

3.2. 5-hafta savollari — dars va laboratoriya savollari model javoblar bilan

S1. Laboratoriya so‘rovi: C dasturlarimizda main roli nima?

Javob. main — kirish nuqtasi: dastur bajarilishi main dan boshlanadi. stdio.h printf ni e’lon qiladi; cs50.h get$_{double}$ (va boshqa CS50 kirgizuvchi yordamchilarini) e’lon qiladi.

S2. Blok oddiy operator bilan qanday farq qiladi va blok ichida nima qilish mumkin?

Javob. Oddiy operator ';' bilan tugaydi. Qavsli qavslar bir nechta operatorlarni murakkab operator (blok) birlashtiradi: u oddiy operator o‘rniga qo‘llanilishi mumkin, bitta butun sifatida kompilyatsiyadan o‘tadi, ichida o‘zgaruvchilar e’lon qilinishi mumkin (ular blok uchun mahalliy), bo‘sh {} bo‘lishi mumkin va yopuvchi qavsdan keyin ';' qo‘yilmaydi. Bloklar bir-biriga ichki-ichki joylashgan.

S3. C shartida «haqiqat» hisoblanadigan nima?

Javob. Shart — ifoda (yoki ifodalar seriyasi), masalan n < 3 yoki x < y || z < y. Har qanday nol bo‘lmagan qiymat ⇒ shart to‘g‘ri, 0 — noto‘g‘ri. Ifoda sonli (yoki ko‘rsatkichli) bo‘lishi kerak; C89/C90 da mantiqiy turi yo‘q, C99 da bool <stdbool.h> orqali mavjud. Misol: if (str) — satr null emas.

S4. else if nima qiladi va bir nechta shartlar to‘g‘ri bo‘lsa nima bo‘ladi?

Javob. else boshqa boshqaruv yo‘llarini qo‘shadi. Shartlar birinchi to‘g‘ri bo‘lguncha ketma-ket tekshiriladi; ichki operator faqat u uchun bajariladi. Agar bir nechta shartlar to‘g‘ri bo‘lsa, faqat birinchi shakl bajariladi. Konstruktsiya ichki-ichki if ga teng.

S5. Ichki-if lardagi else qaysi if ga tegishli va uni qanday o‘zgartirish mumkin?

Javob. Qavslar bo‘lmaganda else eng yaqin ichki if ga bog‘lanadi. Else ni tashqi if ga bog‘lash uchun ichki if ni qavsli qavslarga olish kerak: if (x % 4 == 0) { if (x % 2 == 0) y = 2; } else y = 1;.

S6. switch if-else zinasiga qachon qulayroq va uning cheklanishlari nima?

Javob. switch bir butun sonli (yoki belgili) o‘zgaruvchini turli butun konstant ifodalardan iborat case-yorliqlari bilan solishtiradi; diapazonlar va murakkab shartlar uchun if kerak. default — «aks holda» shakli. Kitobga

ko‘ra, switch uzun if-else zinalaridan samaraliroq.

S7. case da break unutilsa nima bo‘ladi?

Javob. Ijro «pastga tushadi» (falls through) keyingi case ga qiymatni qayta tekshirmaydi. Melodiyada (6-misol) case 2 dan keyin break ni o‘tkazib yuborish kirish uchun A# bo‘lganda C, keyin D ni chiqarishiga olib keldi. Pastga tushish foydali ham bo‘lishi mumkin: case 'Y': case 'y': ikkala belgi uchun bir xil kodni bajaradi.

S8. Melodiya: 'A' belgisi uchun dastur C notasi ni nima chiqaradi?

Javob. Dastur belgi ASCII kodini 7 ga bo‘lgandagi qoldiqni hisoblaydi: 'A' kodi 65; 65 % 7 = 2; case 2 C ni chiqaradi. Jadval: A → 65 → 2 → C.

S9. do-while while dan qanday farq qiladi?

Javob. do- da shart har bir iteratsiyadan keyin hisoblanadi, shuning uchun jism kamida bir marta bajariladi; oxirida albatta nuqta vergul qo‘yiladi: } while (...);. while da shart birinchi hisoblanadi — jism hech qachon bajarilmasligi mumkin.

S10. §6.2.1 dagi ikkita sikl nima chiqaradi va ular qanday farq qiladi?

Javob. Ikkalasi ham 1 2 3 chiqaradi. while versiyasi int i = 1; ni sikldan to‘g‘ri oldin e’lon qiladi; for versiyasi sanagichni sarlavhada e’lon qiladi — u faqat sikl ichida ko‘rinadi. for boshlash, shart va yangilashni bir qatorga birlashtiradi.

S11. break va continue nima qiladi va ijro qayerga o‘tadi?

Javob. break; eng ichki sikl yoki switch dan chiqadi (masalan, c != 'y' bo‘lganda do-while dan oldindan chiqish). continue; eng ichki sikl jismining qolgan qismini o‘tkazib yuborib, uning shartiga o‘tadi; laboratoriya for siklida 'A' bilan continue printf ni chetlab o‘tib i++ yangilashiga, keyin sikl shartiga o‘tadi.

S12. for sarlavhasida uchta ifoda qanday turda va nimalarni qoldirish mumkin?

Javob. Boshlash; shart; inkrement — nuqta vergullar bilan ajratilgan. Har qanday ifoda bo‘sh qoldirilishi mumkin; bo‘sh shart to‘g‘ri deb hisoblanadi. Ifodalar «vergul» operatori orqali murakkab bo‘lishi mumkin: for (i = 1, j = 1; i <= n; j *= i, i++) — vergul pastroq ustuvorlikka ega va chapdan o‘ngga hisoblanadi, bu funksiya argumentlari orasidagi vergul emas.

S13. strlensiz HI! satrining uzunligini qanday qo‘lda hisoblash mumkin?

Javob. Hisobni n = 0 dan boshlang; name[n] != '\0' bo‘lguncha n ni oshiring; oxirgi belgida to‘xtang — terminal hisoblanmaydi. n = 0 H ni o‘qiydi, n = 1 I ni o‘qiydi, n = 2 ! ni o‘qiydi, n = 3 NUL ga yetadi — shart noto‘g‘ri, 3 chiqariladi.

S14. Funktsiya ichidagi statik o‘zgaruvchi nima qiladi va register dan qanday farq qiladi?

Javob. funksiya ichidagi static: o‘zgaruvchi baribir lokal bo‘ladi, lekin dastur ishga tushganda bir marta inicializatsiyalanadi va har bir chaqiruvda qayta inicializatsiyalanmaydi; funksiya tashqarisidagi static ko‘rish sohasini joriy fayl bilan cheklaydi (extern bo‘lishi mumkin emas). register — o‘zgaruvchini registrda saqlash so‘rovi: faqat lokal o‘zgaruvchilar va argumentlar; ortiqcha e’lonlar e’tiborga olinmaydi; register-o‘zgaruvchining manzilini olish mumkin emas.

3.3. 5-hafta testlari

T1. switch ichidagi break nima qiladi?

a) butun dasturni tugatadi

b) switchdan chiqadi; bajarish qavs yopilgandan keyin davom etadi

c) defaultga o‘tadi

d) sikl shartini o‘tkazib yuboradi

Javob: b) switchdan chiqadi; bajarish qavs yopilgandan keyin davom etadi

Tushuntirish: "break leaves the switch" — 'B' case dagi keyin printf keladi, keyin break va dastur switchdan keyin davom etadi; break dasturni tugatmaydi.

T2. do-while jismi necha marta bajarilishiga kafolat beriladi?

a) nol marta
b) aynan bir marta
c) kamida bir marta
d) kompilyatorga bog‘liq

Javob: c) kamida bir marta

Tushuntirish: do-while har bir iteratsiyadan keyin shartni tekshiradi ("condition evaluated after each iteration; body executed at least once").

T3. for (int i = 1; i <= 3; i++) printf("%d ", i); nima chiqaradi?

a) 1 2 3
b) 1 2 3 4
c) 0 1 2 3
d) 3 2 1

Javob: a) 1 2 3

Tushuntirish: §6.2.1 dan while/for solishtirishi — ikkala sikl ham 1 2 3 chiqaradi; keyingi oshirish 4 beradi va test noto‘g‘ri bo‘ladi.

T4. for sarlavhasidagi shartni chetlab o‘tsangiz nima bo‘ladi?

a) kompilyatorda xato
b) jism hech qachon bajarilmaydi
c) shart to‘g‘ri deb hisoblanadi
d) sikl aynan bir marta bajariladi

Javob: c) shart to‘g‘ri deb hisoblanadi

Tushuntirish: "Expressions can be empty (condition assumed to be "true")" — bo‘sh for sharti har doim to‘g‘ri.

T5. continue; nima qiladi?

a) sikldan chiqadi
b) ichki sikl jismining qolgan qismini o‘tkazib yuborib, shartga o‘tadi
c) siklni initssizatsiyadan boshlaydi
d) funksiyadan chiqadi

Javob: b) ichki sikl jismining qolgan qismini o‘tkazib yuborib, shartga o‘tadi

Tushuntirish: continue; ichki sikl jismining qolgan qismini o‘tkazib yuborib, sikl shartiga sakraydi; misoldagi gcd i bo‘lmaydigan bo‘luvchi oddiy o‘tkazib yuboriladi.

T6. 65 % 7 nimaga teng?

a) 1
b) 2
c) 3
d) 9

Javob: b) 2

Tushuntirish: 6.1-misol: 'A' kodi 65, 65 % 7 = 2 — case 2 C notasi chiqaradi.

T7. ichma-ich iflarda qavsli bo‘lmagan else qaysifga tegishli?

a) eng tashqari
b) eng yaqin ichkari
c) fayldagi birinchi
d) hech biriga — bu kompilyatorda xato

Javob: b) eng yaqin ichkari

Tushuntirish: "Nesting if statements" slaydi: else ichki if bilan bog‘lanadi; tashqari bilan bog‘lash uchun qavslar kerak.

T8. while sikli sharti qachon tekshiriladi?

a) jismdan keyin
b) jismdan oldin
c) jismning o‘rtasida
d) faqat birinchi iteratsiyada

Javob: b) jismdan oldin

Tushuntirish: "Condition evaluated first, so body may never be executed" — while jismi hech qachon bajarilmasligi mumkin.

T9. C99 da mantiqiy tur bool ... sarlavhasi orqali mavjud.

a) <stdbool.h>
b) <bool.h>
c) <stdio.h>
d) <logic.h>

Javob: a) <stdbool.h>

Tushuntirish: "in C99, bool type available (use stdbool.h)"; C89/C90 da mantiqiy turi yo‘q — shart oddiy sonli ifoda.

T10. for sarlavhasida bir nechta ifodalarni bog‘lovchi operator qaysi?

a) nuqta-vergul
b) vergul
c) &&
d) ikki nuqta

Javob: b) vergul

*Tushuntirish: "Compound expressions separated by commas… Comma: operator with lowest precedence, evaluated left-to-right; not same as between function arguments" — for (i = 1, j = 1; i <= n; j *= i, i++).*

3.4. Atamalar — 5-hafta glossariysi

Atama	Ma’nosi
Blok / murakkab operator	{ … } — operatorlarni bitta birlikka birlashtiradi; bitta butun sifatida kompilyatsiyalanadi; lokal ko‘rish sohasini yaratadi; } dan keyin nuqta-vergul kerak emas
Tarmoqlanish (selection)	bajarish yo‘lini tanlash: if / else if / else va switch
Shart (condition)	ifoda: nol bo‘lmagan qiymat — to‘g‘ri, 0 — noto‘g‘ri; sonli ifoda yoki ko‘rsatkich
switch	butun sonli (belgili) o‘zgaruvchi bo‘yicha tanlov operatori; mazmuni — blok

Atama	Ma’nosi
case-yorliq	butun sonli doimiy kirish yorlig‘i switch blokiga; yorliqlar har xil
default	switch "aks holda" shoxchasi — hech qanday case mos kelmaganda
Tushib ketish (fall-through)	break bo‘lmaganda keyingi casega bajarishni davolash; 'Y': case 'y': ni birlashtirish uchun ishlatiladi
while	oldin shartli sikl: shart jismdan oldin hisoblanadi; jism hech qachon bajarilmasligi mumkin
for	"hisoblovchi" sikl: initssizatsiya; shart; sarlavhadagi inkrement; bo‘sh shart — to‘g‘ri
do-while	post-shartli sikl: jism kamida bir marta bajariladi; oxirida — nuqta-vergul
break	eng ichki sikl yoki switch dan chiqish
continue	ichki sikl jismining qolgan qismini o‘tkazib yuborib, shartga o‘tish (for da — yangilashga)
Qo‘riqchi qiymat (sentinel)	kirish chegaralovchi belgi: kuynagi '#', do-while misolida '%'
Ko‘lam va soyalanish (scope, shadowing)	scope — o‘zgaruvchan amal qilish hududi (odatda e’lon bloki; global — funktsiyalar tashqarisida); shadowing — ichki e’lon bir xil nom bilan tashqi ni yashiradi (mahalliy nmax = 25 global 20 ni yashiradi)

4. 6-hafta — Funksiyalar va tuzilmalar (Functions and Structures)

Hafta fayllari: Week-6-Functions-and-Structures.txt (laboratoriya «WEEK 6 · CHAPTER 7: From Scratch blocks to a coffee machine») · Lecture$_6$.txt (TashTech Lecture 6 — October 20, 2026)

6-hafta Scratch bloklarini haqiqiy C funktsiyalari bilan bog‘laydi: ta’rif, prototip, chaqiruv, parametrlar va argumentlar, qaytariladigan qiymatlar — va qahva mashinasi ajoyib modeliga tuzilmalar (typedef struct, maydonlar, nuqta) qo‘shadi. 6-ma’ruza boshqaruv konstruksiyalarini (goto va spagetti-kod) mustahkamlaydi va kir-chiqni to‘liq yopadi: standart I/O, formatlashtirish printf/scanf, satrlar, fayllar va buyruq qatori argumentlari. Quyida — haftaning barcha qoidalari, savollar, testlar va terminlari.

4.1. Asosiy tushunchalar («haftaning barcha qoidalari» 6)

Funksiya: ta’rif, prototip, chaqiruv

Funksiya — nomlangan buyruklar to‘plami. void print$_{\text{helloworld}}$(void) sarlavhasini o‘qing: nomdan oldin void — qaytariladigan natija yo‘q; (void) — kirishlar yo‘q; qavsli qavslar — jism. Qaytariladigan qiymat va chiqarilgan chiqishni farqlash muhim.

```
void print_hello_world(void)
{
    printf("Hello, world!\n");
}
// calls inside main
print_hello_world();
print_hello_world();
```

To‘liq dastur: **prototip funktsiyani main chaqirishdan oldin e’lon qiladi** — ';' bilan tugaydi va jismga ega emas. Kodda toping: Prototype — funktsiyani e’lon qilish; Call — hozirroq bajarish; Definition — uning ishini tasvirlash. Dastur chiqishi — Hello, world! ikki marta:

```
#include
```

```
void print_hello_world(void);

int main(void)
{
    print_hello_world();
    print_hello_world();
    return 0;
}

void print_hello_world(void)
{
    printf("Hello, world!\n");
}
```

Agar funktsiya ichidagi xabarni o‘zgartirsangiz, ikkala chaqiruv ham o‘zgartirilgan ta’rifdan foydalanadi: qayta foydalanish — umumiy xulq-atvor, faqat kamroq chop etish emas.

[Manba: Week-6-Functions-and-Structures, slaydlar 5–6]

Parametrlar, argumentlar va qaytariladigan qiymat

Bitta funktsiya boshqasidan foydalanishi mumkin. **Parametr** — funktsiya ichidagi kirish nomi (count); **argument** — chaqiruv orqali uzatilgan qiymat (3). count $\text{print}_{\text{meows}}$ uchun mahalliy; har bir chaqiruvda qiymat nusxalanadi:

```
void print_meow(void)
{
    printf("meow\n");
}

void print_meows(int count)
{
    for (int i = 0; i < count; i++)
    {
        print_meow();
    }
}

// calls inside main
print_meows(1);
print_meows(3);
```

Qaytariladigan qiymat chaqiruvchiga qaytadi. Kitobdagi IntegerRoot misoli — butun sonli ildiz: int — natija turi; number — kirish parametri; chaqiruvchi (caller) — chaqiruvni amalga oshiruvchi funktsiya (bu yerda main). IntegerRoot sonni qaytaradi; **chiqaruvchi ekranga chiqaradi** printf orqali:

```
int IntegerRoot(int number);

// calls inside main
int first = IntegerRoot(27);
int second = IntegerRoot(16);
printf("%d %d\n", first, second);
```

27 uchun: $5 \times 5 = 25 \le 27$; $6 \times 6 = 36 > 27$ — javob 5. 16 uchun — 4. Dastur **5 4** ni chiqaradi.

[Manba: Week-6-Functions-and-Structures, slaydlar 7–8]

Tuzilmalar: typedef struct va maydonlarga kirish

Tuzilma turini ta’riflash main dan yuqorida: struct { … } bog‘liq maydonlarni bir turga birlashtiradi; int water_{ml}; bitta butun maydonni e’lon qiladi; typedef turga qulay nom beradi; Stock — qavs yopilgandan

keyingin tur nomi; barcha e’lon ';' bilan tugaydi. **Maydon (field) — yozuvdagi bitta nomlangan qiymat.**

```
typedef struct
{
    int water_ml;
    int milk_ml;
    int beans_g;
    int paper_cups;
} Stock;
```

O‘zgaruvchi yaratish va maydonlar bilan ishlash: qiymatlar maydonlar tartibida keladi — suv, sut, donlar, stakanlar; bitta maydonga kirish — nuqta orqali. Stock (tur, katta harf) va stock (o‘zgaruvchi, kichik harf) — har xil nomlar:

```
Stock stock = {500, 200, 40, 3};
printf("%d\n", stock.water_ml);
stock.water_ml = 450;
printf("%d\n", stock.water_ml);
```

Maydon	Dastlabki qiymatdan keyin
$water_{ml}$	500
$milk_{ml}$	200
$beans_{g}$	40
$paper_{cups}$	3

dastlabki qiymatdan keyingi stock; dastur chiqishi: 500, keyin 450 — faqat $water_{ml}$ maydoni o‘zgaradi [Manba: Week-6-Functions-and-Structures, slayd 10]

[Manba: Week-6-Functions-and-Structures, slaydlar 9–10]

Yozuvning nusxasi: funktsiya alohida Stock qaytaradi

Yordamchi Stock parametrining **nusxasini** oladi: nusxani o‘zgartiradi va uni qaytaradi; chaqiruvchi natijani qayerda saqlashni hal qiladi:

```
Stock stock_after_water_use(
    Stock stock, int amount_ml)
{
    if (amount_ml >= 0 &&
        amount_ml <= stock.water_ml)
    {
        stock.water_ml -= amount_ml;
    }
    return stock;
}

// Caller · stock starts with 500 ml
Stock preview =
    stock_after_water_use(stock, 100);
printf("%d\n", stock.water_ml);
printf("%d\n", preview.water_ml);
stock =
    stock_after_water_use(stock, 50);
printf("%d\n", stock.water_ml);
```

Vaqt	$stock.water_{ml}$	$preview.water_{ml}$
Birinchi chaqiruvdan keyin	500	400
stock saqlanganidan keyin	450	400

Birinchi chaqiruv o‘zgartirilgan nusxani qaytaradi, asl 500 qoladi; keyingi chaqiruv 50 ml dan boshlanadi va o‘zgarmagan 500 dan boshlanib, 450 ni stock saqlaydi [Manba: Week-6-Functions-and-Structures, slayd 11]

[Manba: Week-6-Functions-and-Structures, slayd 11]

Qahva mashinasi modeli: $can_{makecoffee}$, $stock_{aftercoffee}$ va Cup

Ajoyib model: mashinada suv 500 / sut 200 / qahva donalari 40 / stakanlar 3; bitta latte uchun 100 / 50 / 10 / 1 kerak. **Retsep ham tur-strukturadir (Recipe); o‘zgaruvchi latte bitta ichimlik uchun miqdorlarni saqlaydi; qabul qilingan bitta buyurtma bitta stakan ishlatadi.** Har bir resursni ayiramiz: 400 ml suv, 150 ml sut, 30 g don, 2 stakan.

Yechim funksiyasi: bool — true/false natija; >= — «kamida shu miqdorda»; && — har bir shart bajarilishi kerak; return — natijani chaqiruvchiga uzatadi. «Aynan shuncha» qabul qilinadi; bitta stakan yetishmasa ham buyurtma to‘xtatilishi mumkin:

```
bool can_make_coffee(
    Stock stock, Recipe recipe)
{
    return
        stock.water_ml >= recipe.water_ml &&
        stock.milk_ml >= recipe.milk_ml &&
        stock.beans_g >= recipe.beans_g &&
        stock.paper_cups >= recipe.paper_cups;
}
```

Ushbu latte uchun Stock	Tayyorlash mumkinmi?
500 / 200 / 40 / 3	true
100 / 50 / 10 / 1	true
500 / 200 / 40 / 0	false

Sonlar — suv/sut/qahva donalari/stakanlar [Manba: Week-6-Functions-and-Structures, 13-slaid]

Keyingi Stock hisobi: agar qahva tayyorlash mumkin bo‘lsa — barcha to‘rtta resursni ayirib va nusxasini qaytaramiz; -= ayirib va natijani saqlaydi; return stock; yangilangan nusxani qaytaradi. Ikkita chaqiruvni solishtiring:

```
Stock stock_after_coffee(
    Stock stock, Recipe recipe)
{
    if (can_make_coffee(stock, recipe))
    {
        stock.water_ml -= recipe.water_ml;
        stock.milk_ml -= recipe.milk_ml;
        stock.beans_g -= recipe.beans_g;
        stock.paper_cups -= recipe.paper_cups;
    }
    return stock;
}

stock_after_coffee(stock, latte);
// stock is still unchanged

stock =
    stock_after_coffee(stock, latte);
// stock now keeps the returned record
```

Chaqiruvchining Stocki	Suv / sut / qahva donalari / stakanlar
Buyurtmadan oldin	500 / 200 / 40 / 3

Chaqiruvchining Stocki	Suv / sut / qahva donalari / stakanlar
Natijani saqlaganidan keyin	400 / 150 / 30 / 2

Birinchi chaqiruv qaytarilgan nusxani tashlaydi, ikkinchisi saqlaydi [Manba: Week-6-Functions-and-Structures, 14-slaid]

Buyurtma bloki: $\text{print}_{\text{stock}}$ va $\text{print}_{\text{cup}}$ ma'lumotlarni ko‘rsatadi; $\text{create}_{\text{cup}}$ ichimlik yozuvini (Cup) quradi; false «ko‘mirsiz» degan ma'noni anglatadi. Agar zaxiralar bo‘lmasa ($\text{stock.paper}_{\text{cups}} = 0$), if sharti hech qanday tayinlashni amalga oshirmaydi — barcha joriy maydonlar stock va avvalgi Cup o‘zgarmaydi:

```
print_stock(stock);
if (can_make_coffee(stock, latte))
{
    stock =
        stock_after_coffee(stock, latte);
    cup = create_cup(latte, false);
}
print_stock(stock);
print_cup(cup);
```

[Manba: Week-6-Functions-and-Structures, 12–15-slaidlar]

goto, spagetti-kod va xatolarni ishlash

kodning istalgan joyiga **shartli o‘tishga imkon beradi**; joy **belgisi (label)** bilan belgilanadi — o‘zgaruvchi nomiga o‘xshash nomli nuqta, undan keyin ':' keladi.

```
start:
{
    if (cond)
        goto outside;
    /* some code */
    goto start;
}
outside:
/* outside block */
```

Dijkstra, «Go To Statement Considered Harmful» (Communications of the ACM 11(3), 1968): ortiqcha goto **spagetti-kod** yaratadi — kodni o‘qish va xatolarni tuzatish qiyinlashadi; goto bilan yozilgan har qanday kod uni ishlatmasdan qayta yozish mumkin. Lekin C da da'vo bor: C++ va Java da xatolardan keyin tiklanish uchun istisno mexanizmi mavjud, C da esa **goto — bu ichki bloklardan chiqishning qulay usuli** (error: belgisiga o‘tish ikkala blokni ham o‘tkazib yuboradi).

[Manba: Lecture$_{6}$, 5–7-slaidlar]

Kirish-chiqarish: putchar/getchar, printf va uning formatlari, scanf

Kirish-chiqarish vositalari **<stdio.h>** standart kutubxonasi beradi, til emas. **Matn oqimi** — '\n' bilan tugaydigan qatorlar seriyasi (kutubxana o‘zi '\r\n'−'\n' ga aylantiradi); **ikilik oqim** — xom baytlar seriyasi; kutubxona oqimlari buferlangan.

int putchar(int) belgi c ni standart chiqarishga joylaydi; xatolikda bosib chiqarilgan belgi yoki EOF ni qaytaradi. **int getchar()** standart kirishdagi keyingi belgini yoki xatolikda EOF ni qaytaradi. Klassik misol — kichik harfga aylantirish (c = c − 'A' + 'a'; — bu char butun son aylantirishiga bog‘liq):

```
int main()
{
    char c;
    while ((c = getchar()) != EOF)
    {
        if (c >= 'A' && c <= 'Z')
            c = c - 'A' + 'a';
```

```
        putchar(c);
    }
    return 0;
}
```

Standart kirish o‘rniga fayldan o‘qish uchun '<' operatori ishlatiladi (*nix): ./a.out < file.txt — bu til emas, OS imkoniyati.

int printf(char format[], arg1, arg2, ...) — formatlangan chiqarish: o‘zgaruvchan sonli argumentlarni qabul qiladi, bosib chiqarilgan belgilar sonini qaytaradi. Formatda so‘zma-so‘z matn va % bilan boshlanadigan maxsus belgilar mavjud: **%[flags][width][.precision][length]<type>**.

Tur	Ma’no	Misol
d, i	butun son	printf("%d",10); / *10 ni chiqaradi* /
u	belgili bo‘lmagan butun son	printf("%u",10); / *10 ni chiqaradi* /
x, X	butun son (hex)	printf("%x",10); / *0xa ni chiqaradi* /
c	belgi	printf("%c",'A'); / *A ni chiqaradi* /
s	satr	printf("%s","hello"); / *hello ni chiqaradi* /
f	haqiqiy son	printf("%f",2.3); / *2.3 ni chiqaradi* /
e, E	haqiqiy son (eksponensial)	1e3, 1.2E3, 1E-3
%	so‘zma-so‘z %	printf("%d %%",10); / *10% ni chiqaradi* /

printf tur maxsus belgilari [Manba: Lecture$_{6}$, 12-slaid; Lecture$_{71}$, 3-slaid]

Modifikator	Ma’no
h	short sifatida talqin qilish; i, d, o, u, x bilan ishlash
l	long sifatida talqin qilish; i, d, o, u, x bilan ishlash
L	double sifatida talqin qilish; e, f, g bilan ishlash

Uzunlik modifikatorlari [Manba: Lecture$_{6}$, 16-slaid]

Chaqiruv	Chiqarish
printf("%d",10)	«10»
printf("%4d",10)	«bb10» (b — bo‘shliq: width)
printf("%7s","hello")	«bbhello»
printf("%d,%+d,%+d",10,-10)	«10,+10,-10» (flag +)
printf("%04d",10)	«0010» (0 bayrog‘i)
printf("%-7s","hello")	«hellobb» (– bayrog‘i: chap chegaraga)
printf("%.2f,%.0f",1.141,1.141)	«1.14,1» (aniqlik)
printf("%.2e,%.0e",1.141,100.00)	«1.14e+00,1e+02»
printf("%.4s","hello")	«hell»
printf("%.1s","hello")	«h»

kenglik, bayroqlar va aniqlik misollarida [Manba: Lecture$_{6}$, slaydlar 13–15]

int scanf(char *format, ...) — kirish analogi printf: standart kirish belgilarini o‘qiydi, ularni format spetsifikatsiyasiga (printfdagi bilan bir xil) bo‘yicha talqin qiladi; o‘zgaruvchan miqdordagi argumentlarni qabul qiladi; bir nechta elementlarni o‘qiyotganda har biri bo‘shliq belgilariga ajratilgan deb hisoblanadi; **scanf**

bo‘shliq belgilarini e’tiborsiz qoldiradi; o‘qilgan elementlar sonini yoki EOF ni qaytaradi. Muhim: **argumentlar o‘zgaruvchilar manzillariga (ko‘rsatkichlarga) bo‘lishi kerak**. Moslik misollari:

printf	scanf
printf("%d",x)	scanf("%d",&x)
printf("%f",f)	scanf("%f",&f)
printf("%s",str)	scanf("%s",str) — & kerak emas
printf("%s",str)	scanf("%20s",str) — & kerak emas
printf("%s %s",fname,lname)	scanf("%20s %20s",fname,lname)

printf ↔ scanf [Manba: Lecture$_6$, slayd 20]

Formatlangan ma’lumotlarni massivlarga yozish va o‘qish mumkin: **int sprintf(char string[], char format[], arg1, arg2)** — printfga o‘xshash, lekin chiqish stringga yoziladi (o‘lchami tekshirilmaydi); **int sscanf(char str[], char format[], arg1, arg2)** — scanfga o‘xshash, lekin kirish str dan o‘qiladi; ikkalasi ham yozilgan/o‘qilgan elementlar sonini yoki xatoda manfiy qiymatni qaytaradi.

[Manba: Lecture$_6$, slaydlar 8–21]

Satrlar massiv sifatida, strcmp, fayllar va buyruq qatori

Satrlar belgilar massivlari sifatida ifodalanadi; C satr uzunligini cheklamaydi — satr oxiri nol bayt bilan belgilanadi: "hello" — bu {'h','e','l','l','\0'} massivi. E’lonlar:

```
char str[] = "hello";   /* compiler takes care of size */
char str[10] = "hello"; /* make sure the array is large enough */
char str[] = {'h','e','l','l',0};
```

Eslatma: satrda " belgisini bo‘lishi uchun \" dan foydalaning. Satrlarni solishtirish uchun **<string.h>** sarlavhasi **int strcmp(char s[], char t[])** ni beradi — lug‘at tartibida solishtirish (kichik harflar katta harflardan keyin keladi): agar s t dan oldin bo‘lsa <0 qiymatini qaytaradi; satrlar bir xil bo‘lsa 0; agar s t dan keyin bo‘lsa >0; strcmp registrga sezgir. Misollar: strcmp("A","a") → <0; strcmp("IRONMAN","BATMAN") → >0; strcmp("aA","aA") → ==0; strcmp("aA","a") → >0.

Fayllar: FILE *fopen(char name[], char mode[]) oqimni ochadi; rejimlar "r" (o‘qish), "w" (yozish), "a" (qo‘shimcha), boshqalar orasida; "b" ikkilik fayllar uchun qo‘shiladi; fopen oqimga ko‘rsatkich yoki NULL ni qaytaradi. int fclose(FILE* fp) oqimni yopadi (OS resurslarini bo‘shatadi); dastur tugaganda barcha ochilgan fayllar uchun fclose avtomatik chaqiriladi. Standart kirish va chiqish ham FILE *(stdin, stdout); stderr — standart chiqish xatolari uchun alohida oqim. Belgilar bo‘yicha: int getc(FILE* fp)* (getchar — bu getc(stdin): #define getchar() getc(stdin)); int putc(int c, FILE *fp) (putchar — putc(c, stdout)); satr bo‘yicha: char fgets(char line[], int maxlen, FILE *fp) bir satrni maxlen belgigacha (qator o‘zgaruvchisini o‘z ichiga olgan) o‘qiydi, massivga ko‘rsatkich yoki oqim oxirida NULL ni qaytaradi; int fputs(char line[], FILE* fp) satrni yozadi, muvaffaqiyatda 0 yoki EOF ni qaytaradi; int fscanf(FILE *fp, char format[], ...) — scanf/sscanfga o‘xshash, lekin fp oqimidan.

Buyruq qatori argumentlari: main parametlarni qabul qilishi mumkin, ular dastur ishga tushirilganda to‘ldiriladi: **int main(int argc, char *argv[])**; argc — argumentlar hisoblagichi; argv[] — har bir argumentga ko‘rsatkichlar massivi; argumentlar dastur nomini ham o‘z ichiga oladi: ./cat a.txt b.txt argc = 3, argv[0] = "cat", argv[1] = "a.txt", argv[2] = "b.txt" beradi; ./cat argc = 1, argv[0] = "cat" beradi.

[Manba: Lecture$_6$, slaydlar 17–27]

4.2. 6-hafta savollari — ma’ruza va laboratoriya savollari model javoblar bilan

S1. Funktsiya nomi oldidagi va qavs ichidagi (void) nima degani?

Javob. Nom oldidagi void — qaytaruvchi natija yo‘q; (void) — kirishlar yo‘q. void $\text{print}_{\text{helloworld}}$(void) da ikkala ham o‘z o‘rnida; qavsli qavslar tanani o‘rab oladi. Qaytaruvchi qiymat chop etilgan chiqindidan farqlanishi kerak.

S2. Prototip chaqiruv va ta’rifdan qanday farq qiladi?

Javob. Prototip maindan oldin funksiyani e’lon qiladi: void $\text{print}_{\text{helloworld}}$(void); — nuqta-vergul bilan tugaydi va tanaga ega emas. Chaqiruv — $\text{print}_{\text{helloworld}}$(); — funksiyani hozirroq bajaradi. Ta’rif — qavsli qavsli tanis — uning ishini tasvirlaydi. Bir dasturda: prototip maindan oldin, chaqiruvlar main ichida, ta’rif keyin.

S3. Darsni takrorlash-savol: for (int i = 1; i <= 3; i++) printf("%d ", i); nima chiqaradi?

Javob. 1 2 3 — hisoblovchi 1, 2, 3 dan o‘tadi; i = 4 da i <= 3 sharti noto‘g‘ri. 5-haftadagi while/for solishtiruvi xuddi shu natija beradi.

S4. Darsni takrorlash-savol: int streak = 2; int practised = 0; if (practised == 1) streak++; else streak = 0; dan keyin streak qanday qiymatga ega?

Javob. streak = 0: practised == 1 noto‘g‘ri (practised 0 ga teng), else sharti bajariladi va streak nolga qo‘yiladi. Har qanday noldan farqli qiymat — to‘g‘ri, 0 — noto‘g‘ri.

S5. Darsni takrorlash-savol: birinchi break bo‘lmaganda score qanday qiymatga ega? int score = 0; char reaction = 'W'; switch (reaction) { case 'W': score += 2; case 'L': score -= 1; break; }

Javob. score = 1. 'W' belgisi 'W' case bilan mos keladi: score += 2 2 beradi; break yo‘q — bajarilish 'L' case ga o‘tadi yangi tekshiruvsiz: score -= 1 1 beradi; switch tugatadi.

S6. Parametr argumentdan qanday farq qiladi?

Javob. Parametr — funksiya ichidagi kirish nomi: count void $\text{print}_{\text{meows}}$(int count); da bu nom funksiya uchun mahalliy. Argument — chaqiruv beradigan qiymat: $\text{print}_{\text{meows}}$(3) dagi 3. Har bir chaqiruvda qiymat parametrga nusxalanadi.

S7. IntegerRoot(27) va IntegerRoot(16) chaqiruv juftligi nima chiqaradi?

Javob. 5 4. 27 uchun: $5 \times 5 = 25 \le 27$, $6 \times 6 = 36 > 27$, shuning uchun butun sonli ildiz 5; 16 uchun — 4. IntegerRoot sonni qaytaradi, uni printf("%d %d\n", first, second); orqali chaqiruvchi chiqaradi.

S8. Maydon (field) nima va Stock stockdan qanday farq qiladi?

Javob. Maydon — yozuvdagi (water_{ml}, milk_{ml}, beans_{g}, $\text{paper}_{\text{cups}}$) bitta nomlangan qiymat. Stock — qavs yopilgandan keyin typedef bilan kiritilgan tur nomi; stock — bu turdagi o‘zgaruvchi. Registrani ahamiyati bor: Stock va stock — turli nomlar.

S9. Stock preview = $\text{stock}_{\text{afterwateruse}}$(stock, 100); dan keyin stockdagi suv hali ham 500, previewda esa 400 bo‘lishining sababi nima?

Javob. Funksiyadagi stock parametri — nusxa: yordamchi nusxadan 100 ayiradi va o‘zgartirilgan nusxani qaytaradi; main dagi asl 500 da qoladi. Chaqiruvchi natijani qayerda saqlashni tanlaydi: preview 400 ni oladi. stock = $\text{stock}_{\text{afterwateruse}}$(stock, 50); dan keyin — yangi chaqiruv o‘zgarmagan 500 dan boshlanadi va 450 ni stockga saqlaydi.

S10. stock = $\text{stock}_{\text{aftercoffee}}$(stock, latte); tayinlash nima o‘zgartiradi va uni bo‘lmagida nima bo‘ladi?

Javob. Tayinlash bilan stock qaytarilgan yozuvni saqlaydi: 400 / 150 / 30 / 2. Tayinlanmasdan (faqat $\text{stock}_{\text{aftercoffee}}$(stock, latte);) qaytarilgan nusxa tashlanadi — stock o‘zgarmaydi: "stock is still unchanged".

S11. stock.$\text{paper}_{\text{cups}}$ = 0 bo‘lganda buyurtma bajarilmaydi va hech narsa o‘zgarmaydigan sababi nima?

Javob. $can_{makecoffee}$ barcha to‘rtta resurs talab qilinadiganidan kam bo‘lmasligini talab qiladi (&& — har bir shart bajarilishi kerak): bitta yetishmayotdan stakanchik yetarli bo‘ladi false qaytarish uchun. if shakli hech qanday tayinlamani bajarmaydi — barcha stock maydonlari va oldingi cup (latte) o‘zgarmaydi.

S12. goto xavfli va C da u qachon oqlanadi?

Javob. ortiqcha goto spagetti-kod yaratadi (Dijkstra, "Go To Statement Considered Harmful", CACM 11(3), 1968): kodni o‘qish va xatolarni tuzatish qiyin; har qanday goto kodli uni bilan yozmasdan qayta yozish mumkin. C da goto xatolarni qayta ishlashda chuqur bloklardan chiqishning qulay usuli sifatida oqlanadi — C++/Java da bunday uchun istisnolar mavjud, C da yo‘q.

S13. scanf & ni talab qiladi va qachon kerak emas?

Javob. scanf argumentlari o‘zgaruvchilar manzillari (ko‘rsatkichlar) bo‘lishi kerak: scanf("%d", &x) o‘qilgan sonni x manziliga yozadi. Belgilar massivlari (satrlar) uchun massiv nomi allaqachon manzildir: scanf("%s", str) — &siz; shunday qilib scanf("%20s", str) va scanf("%20s %20s", fname, lname). scanf bo‘sh belgilarni e’tiborsiz qoldiradi va o‘qilgan elementlar sonini yoki EOF ni qaytaradi.

S14. argc va argv nima saqlaydi?

Javob. int main(int argc, char *argv[]): argc — argumentlar soni; argv[] — har bir argumentga ko‘rsatkichlar massivi; argumentlar dastur nomini o‘z ichiga oladi. Misol: ./cat a.txt b.txt argc = 3, argv[0] = "cat", argv[1] = "a.txt", argv[2] = "b.txt" beradi; yakka ./cat argc = 1 beradi.

4.3. 6-hafta testlari

T1. Funksiya prototipi nima?

a) figurali qavslar bilan funksiya tanasi
b) funksiya e’loni, ';' bilan tugaydi va tanasiz
c) main dan funksiya chaqiruvi
d) funksiya ichidagi o‘zgaruvchi

Javob: b) funksiya e’loni, ';' bilan tugaydi va tanasiz

Tushuntirish: "A prototype declares a function before main calls it. It ends with ; and has no body" — prototip e’lon qiladi, chaqiruv amalga oshiradi, ta’rif ishni tasvirlaydi.

T2. void $print_{meow}$(void) funksiyasida nechta kirish va qaytariladigan qiymatlar bor?

a) 1 kirish, 1 qiymat
b) kirish yo‘q, qaytariladigan qiymat yo‘q
c) kirish yo‘q, 1 qiymat
d) 1 kirish, qiymat yo‘q

Javob: b) kirish yo‘q, qaytariladigan qiymat yo‘q

Tushuntirish: void nomdan oldin — qaytariladigan natija yo‘q; (void) — kirishlar yo‘q.

T3. IntegerRoot(27) qaytaradi?

a) 4
b) 5
c) 6
d) 27

Javob: b) 5

Tushuntirish: $5 \times 5 = 25 \le 27$, $6 \times 6 = 36 > 27$ — 27 ning butun sonli kvadrat ildizi 5; ikkita chaqiruv 5 4 ni chiqaradi.

T4. stock o‘zgaruvchisining $water_{ml}$ maydoniga qanday murojaat qilish kerak?

a) $stock.water_{ml}$
b) $Stock.water_{ml}$
c) $stock->water_{ml}$
d) $stock:water_{ml}$

Javob: a) $stock.water_{ml}$

Tushuntirish: bitta maydonga kirish nuqta orqali: $stock.water_{ml} = 450$; (Stock — bu tur nomi, o‘zgaruvchi emas).

T5. typedef struct { … } Stock; da Stock nomi — bu…

a) o‘zgaruvchi
b) typedef tomonidan kiritilgan tur nomi
c) struktura maydoni
d) funksiya

Javob: b) typedef tomonidan kiritilgan tur nomi

Tushuntirish: typedef turga qulay nom beradi; Stock qavs yopilgandan keyin keladi — bu tur nomi, o‘zgaruvchi yoki struct-teg emas.

T6. Stock preview = $stock_{afterwateruse}$(stock, 100); dan keyin $stock.water_{ml}$ qanchaga teng?

a) 400
b) 500
c) 450
d) 0

Javob: b) 500

Tushuntirish: funksiyadagi stock parametri — nusxa; birinchi chaqiruv o‘zgartirilgan nusxani qaytaradi ($preview.water_{ml} = 400$), asl nusxa 500 da qoladi.

T7. printf("%4d", 10); nima chiqaradi?

a) "10"
b) "bb10" (b — bo‘sh joy)
c) "0010"
d) "10bb"

Javob: b) "bb10" (b — bo‘sh joy)

Tushuntirish: width 4 sonni chap tomondan bo‘sh joylar bilan to‘ldiradi; "0010" 0 bayrog‘ini beradi (%04d), "hellobb" minus bayrog‘ini beradi (%-7s).

T8. printf("%.4s", "hello"); nima chiqaradi?

a) hello
b) hell
c) h
d) ello

Javob: b) hell

Tushuntirish: satrlar uchun aniqlik chiqarishni 4 belgigacha qisqartiradi; %.1s "h" beradi, %.2f verguldan keyin ikkita belgi beradi.

T9. x o‘zgaruvchisiga butun son to‘g‘ri qanday o‘qish kerak?

a) scanf("%d", x);
b) scanf("%d", &x);

c) scanf("%d", *x);

d) scanf(%d, &x);

Javob: b) scanf("%d", &x);

Tushuntirish: scanf argumentlari o‘zgaruvchilar manzillari (pointerlar) bo‘lishi kerak; massiv uchun char nomi allaqachon manzil: scanf("%s", str).

T10. fopen funksiyasining "r" rejimi nima qiladi?

a) faqat yozish uchun faylni ochadi

b) faqat o‘qish uchun faylni ochadi

c) faylni qo‘shish uchun ochadi

d) faylni ikkilik qiladi

Javob: b) faqat o‘qish uchun faylni ochadi

Tushuntirish: "r" — faqat o‘qish, "w" — faqat yozish, "a" — qo‘shish; "b" ikkilik fayllar uchun qo‘shiladi; fopen oqat mavjud bo‘lmasa NULL qaytaradi.

4.4. Atamalar — 6-hafta glossariysi

Atama	Ma’nosi
Funksiya (function)	nomlangan ko‘rsatmalar to‘plami; void $print_{helloworld}$(void) — ta’rif, $print_{helloworld}$(); — chaqiruv
Prototip (prototype)	main dan oldin funksiyani e’lon qilish: tur nomi(parametrlar); — ';' bilan tugaydi va jismga ega emas
Ta’rif va chaqiruv	ta’rif — funksiya jymi qavs ichida ("ishni tasvirlang"); chaqiruv — funksiyani nomi bilan ishga tushirish ("hozir ishlatish"): $print_{helloworld}$();
Parametr (parameter)	funksiya ichidagi kirish nomi: count void $print_{meows}$(int count) da
Argument (argument)	chaqiruv orqali uzatiladigan qiymat: 3 $print_{meows}$(3) da
Qaytariladigan qiymat	funksiya chaqiruvchiga yuboradigan natija: int IntegerRoot(int number)
Struktura (structure)	maydonlarni bitta yozuvda birlashtiruvchi tur: struct { … }
Maydon (field)	yozuvdagi bitta nomlangan qiymat: $water_{ml}$, $milk_{ml}$, $beans_{g}$, $paper_{cups}$
typedef	struktura turiga qulay nom beradi: typedef struct { … } Stock; shu turdagi o‘zgaruvchi — yozuv (record): Stock stock = {500, 200, 40, 3}; maydonlarga nuqta orqali kirish
Etiketka (label)	kodning nomlangan nuqtasi goto uchun: nom + ':' (start:, outside:, error:)
Spagetti-kod	juda ko‘p tufayli o‘qilmaydigan, xatolarni topish qiyin kod (Dijkstra, 1968)
Matn oqimi (text stream)	'\n' bilan tugaydigan satrlar seriyasi; '\r\n' o‘zgaruvchini kutubxona o‘zi qiladi; oqimlar buferlanadi
Format belgisi	%[flags][width][.precision][length]<type>: %d, %4d, %04d, %.4s, %e, %%
Komanda satridagi argumentlar	int main(int argc, char *argv[]): argc — hisoblagich, argv — argumentlarga ko‘rsatgichlar massivi, dastur nomini ham o‘z ichiga oladi

5. 7-hafta — Ko‘rsatkichlar, satrlar va rekursiya (Pointers, Strings, Recursion)

Hafta fayllari: Lecture$_{7I}$.txt (TashTech Dars 7 — 2026-yil 27-oktabr: Ko‘rsatkichlar va Xotira Manzillari · Massivlar va Ko‘rsatkich Aritmetikasi · Satrlar · Qidirish va Tartiblash) · Recursion-Lecture.txt (7-HAFTA · 8-BO‘LIM: Rekursiya)

7-hafta — ikki mavzuli. Dars 7 ko‘rsatkichlarni beradi: o‘zgaruvchilar manzillari, jismoniy va virtual xotira, ishlatish, bir nechta chiqadigan funksiyalar, massivlar va manzir arifmetikasi, satrlar va string.h funksiyalari, shuningdek asosiy qidirish va tartiblash algoritmlari (linear search, insertion sort, quicksort, binary search). 8-bob bo‘yicha laboratoriya rekursiyani o‘rgatadi: asosiy holat, kichikroq masala, progress; faktorialni chaqirish steki bo‘yicha kuzatish, fibonachchi va Xanoy minorasi. Quyida — haftaning barcha qoidalari, savollari, testlari va terminlari.

5.1. Asosiy tushunchalar (7-haftaning "barcha qoidalari")

Ko‘rsatkich: manzil (&) va ishlatish (*)

Ko‘rsatkich — bu xotiradagi o‘zgaruvchining manzili; manzilni o‘zgaruvchiga istalg joydan kirish va uni o‘zgartirish uchun foydalansa bo‘ladi — bu ma’lumotlar tuzilishlari uchun juda foyali (va kodni chalkashtirib yuborish bilan mashhur). **Xotiradagi har bir o‘zgaruvchining manzili bor.** Manzili bo‘lmaydi: register-o‘zgaruvchilar; konstantalar/literalar/preprotsessor defineslari; ifodalar (agar natija o‘zgaruvchi bo‘lmasa). Manzilni **&** operatori oladi; t turidagi o‘zgaruvchining manzili **t *** turiga ega:

```
int n = 4;
double pi = 3.14159;
int *pn = &n;      /* address of integer n */
double *ppi = π /* address of double pi */
```

Ishlatish/ko‘rsatish — * operatori: manzil bo‘yicha o‘zgaruvchiga kirish; ishlatilgan ko‘rsatkich oddiy o‘zgaruvchiga o‘xshash harakat qiladi. **Nol ko‘rsatkich** — 0 (NULL): hech narsaga ishlamaydigan ko‘rsatkich:

```
/* prints "pi = 3.14159\n" */
printf("pi = %g\n", *ppi);

/* pi now equals 7.14159 */
*ppi = *ppi + *pn;
```

Ko‘rsatkichlarni istalgan ko‘rsatkich turiga aniq aylantirsa bo‘ladi (void * ga/undan noaniq aylantirish ham mumkin); ishlatilgan ko‘rsatkichning yangi turi haqiqiy ma’lumotlardan qat’iy nazar; bu segmentatsiya xatolariga va boshqa qiyinchilik bilan ushlanadigan xatolarga olib kelishi mumkin:

```
ppi = (double *)pn; /* pn originally of type (int *) */
```

[Manba: Lecture$_{7I}$, slaydlar 5, 9–11]

Xotira: jismoniy va virtual, stek va xira

Jismoniy xotira — bu ma’lumotlar haqiqatda saqlanadigan resurslar: cache, RAM, qattiq disk, olinadigan uskunalar; ularda turli hajmlar va kirish tezliklari; xotirani boshqarish — OSning asosiy funktsiyalaridan biri (u ma’lumotlarni bajarish paytida ko‘chiradi); ichki protsessorlarda xotira juda cheklangan bo‘lishi mumkin. **Virtual xotira** — OSning abstraksiyasi: kodga kirish mumkin bo‘lgan manzil maydoni (32-bitli OSda < 4 GB: odatda Windows uchun 2 GB, Linux uchun 3–4 GB); virtual xotira jismoniy xotiraning turli qismlariga xaritalanadi.

Virtual xotiraning ishlatiladigan qismlari: stek va xira. Stek (stack) — e’lon qilingan o‘zgaruvchilar joylashadi; xira (heap) — dinamik xotira joylashadi.

Funksiyalardagi ko‘rsatkichlar: bir nechta chiqadigan qiymatlar va almashtirish

Funksiyalar bittadan ortiq qiymat qaytarmaydi. Kengaytirilg Evklid algoritmi gcd(a, b), x va y (ax + by = gcd) ni qaytaradi — global o‘zgaruvchilar o‘rniga **bir nechta chiqadigan qiymatlar uchun ko‘rsatkichlarni**

o‘tkazish mumkin:

```
int ext_euclid(int a, int b, int *x, int *y);

int x, y, g;
/* assume a, b declared previously */
g = ext_euclid(a, b, &x;, &y;);
```

Funksiya **chaqiruvchi o‘zgaruvchilarni o‘zgartira olishi** uchun unga ularning ko‘rsatkichlari beriladi. Klassik almashtirish:

```
void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}

int a = 5, b = 7;
swap(&a;, &b;);
/* now, a = 7, b = 5 */
```

Tegishli tuzoq — **ko‘rish doirasidan chiqayotgan o‘zgaruvchi**: lokal massivga ko‘rsatkich qaytarish mumkin emas, funksiyaning stek kadri yo‘qolgach ko‘rsatkich yaroqsiz bo‘lib qoladi (kompilyatorlar odatda ogohlantiradi):

```
#include

char * get_message() {
    char msg[] = "Aren't pointers fun?";
    return msg;
}

int main(void) {
    char *string = get_message();
    puts(string);
    return 0;
}
```

[Manba: Lecture$_{71}$, slaydlar 12–14]

Massivlar, sizeof va manzir arifmetikasi

Primitiv massivlar C da uzluksiz xotira blokiga ko‘rsatkich sifatida amalga oshirilgan. arr element 0 ga ko‘rsatkich sifatida harakat qiladi — lekin ko‘rsatkichdan farqli o‘laroq, uni qayta tayinlab bo‘lmaydi:

```
int arr[8];
int a = arr[0];
int *pa = arr;  /* xuddi shu, int *pa = &arr;[0]; */
```

sizeof(): primitiv turlar/o‘zgaruvchilar uchun — turining baytlardagi hajmi; primitiv massivlar uchun — butun massivning baytlardagi hajmi. Massiv uzunligi — makro:

```
int s = sizeof(char);  /* == 1 */
double f;              /* sizeof(f) == 8  (64-bit OS) */
int arr[8];            /* sizeof(arr) == 32 (64-bit OS) */
long arr[5];           /* sizeof(arr) == 40 (64-bit OS) */

/* needs to be on one line when implemented */
#define array_length(arr) (sizeof(arr) == 0 ? \
    0 : sizeof(arr) / sizeof((arr)[0]))
```

Manzir arifmetikasi: ko‘rsatkich int emas, lekin unga butun son qo‘shish/ayirish mumkin: **pa + i arr[i] ga ishora qiladi**; manzil qiymati i, tur hajmiga ko‘paytirilganda ortadi. Agar arr[0] manzili 100 (int) bo‘lsa, arr[3]

manzili 112 (3 × 4 bayt). Slayd savoli: agar char *pc* = *(char*)pa; bo‘lsa, (int *)(pc + i) == pa + 3 **i = 12** bo‘lganda (3 int = 12 bayt har bir char uchun).

[Manba: Lecture$_{71}$, slaydlar 15–17]

Satrlar va string.h kutubxonasi

Satrlar **nol bayti bilan tugaydigan belgilar massivi** sifatida saqlanadi (oxirgi belgi == '\0'). Satr bilan massiv sifatida ishlanadi:

```
char str[] = "This is a string.";
char *pc = str;

*(pc+10) = 'S';
puts(str); /* prints "This is a String." */
```

<string.h> standart bosh funksiyalari: nusxalash — strcpy(), strncpy(); solishtirish — strcmp(), strncmp(); uzunlik — strlen(); birlashtirish — strcat(), strncat(); belgini qidirish — strchr(), strrchr(). Aniq imzolar:

- char *strcpy(strto, strfrom); — strfrom ni strto ga nusxalaydi
- char *strncpy(strto, strfrom, n); — strfrom dan n ta belgini strto ga nusxalaydi
- int strcmp(str1, str2); — str1 va str2 ni solishtiradi: 0, agar teng; musbat, agar str1 > str2; manfiy, agar str1 < str2
- int strncmp(str1, str2, n); — str1 va str2 ning birinchi n ta belgisini solishtiradi
- int strlen(str); — str ning uzunligini qaytaradi
- char *strcat(strto, strfrom); — strfrom ni strto oxiriga qo‘shadi
- char *strncat(strto, strfrom, n); — strfrom dan n ta belgini strto oxiriga qo‘shadi
- char *strchr(str, c); — str da c belgisini qidiradi: birinchi uchrashishiga ko‘rsatgich yoki NULL
- char *strrchr(str, c); — c belgisining oxirgi uchrashishiga ko‘rsatgich yoki NULL

[Manba: Lecture$_{71}$, slaydlar 18–20]

Qidirish va tartiblash: linear, insertion, quick, binary

Asosiy algoritmlar ko‘rsatkichlardan ajoyib foydalanadi; bu faqat bir necha misollar, algoritmlar kursi emas; murakkablik **Big-O yozuvi** bilan tavsiflanadi. Chiziqli qidirish topilgan elementga ko‘rsatgichni yoki NULL ni qaytaradi:

```
int arr[100]; /* array to search */

int * linear_search(int val) {
    int *parr, *parrend = arr + array_length(arr);
    for (parr = arr; parr < parrend; parr++) {
        if (*parr == val)
            return parr;
    }
    return NULL;
}
```

Oddiy joylashtirish tartiblashi — **O(n^2)**: massiv bo‘ylab yurish, to‘g‘ri joyda bo‘lmagan element topilguncha; uni to‘g‘ri pozitsiyaga joylashtirish; massiv oxirigacha takrorlash. Ikkita funksiyaga bo‘lingan:

```
/* move previous elements down until insertion point reached */
void shift_element(unsigned int i) {
    int ivalue;
    /* guard against going outside array */
    for (ivalue = arr[i]; i && arr[i-1] > ivalue; i--)
        arr[i] = arr[i-1]; /* move element down */
    arr[i] = ivalue;       /* insert element */
```

```
}

/* iterate until out-of-order element found;
   shift the element, and continue iterating */
void insertion_sort(void) {
    unsigned int i, len = array_length(arr);
    for (i = 1; i < len; i++)
        if (arr[i] < arr[i-1])
            shift_element(i);
}
```

Tezkor tartiblash usullari (shellsort, mergesort, quicksort…): quicksort — o‘rtacha holatda O(n log n), yomon holatda $O(n^2)$; asosiy elementni (pivot) tanlash; kichikroq elementlarni bir tomonga, kattalarini ikkinchi tomonga ko‘chirish; tomonlarni alohida tartiblash (**rekursiv algoritm**); standart kutubxonada stdlib.h da qsort() sifatida amalga oshirilgan. K&R dan amalga oshirish:

```
void quick_sort(unsigned int left, unsigned int right) {
    unsigned int i, mid;
    int pivot;
    if (left >= right)
        return; /* nothing to sort */
    /* pivot is midpoint; move to left side */
    swap(arr + left, arr + (left + right) / 2);
    pivot = arr[mid = left];
    /* separate into side < pivot (left+1 to mid)
       and side >= pivot (mid+1 to right) */
    for (i = left + 1; i <= right; i++)
        if (arr[i] < pivot)
            swap(arr + ++mid, arr + i);
    /* restore pivot position */
    swap(arr + left, arr + mid);
    /* sort two sides */
    if (mid > left)
        quick_sort(left, mid - 1);
    if (mid < right)
        quick_sort(mid + 1, right);
}

quick_sort(0, array_length(arr) - 1);
```

Quicksort haqida muhokama: hozirgi shaklda beqaror (teng elementlar o‘rin almashishi mumkin); joyida tartiblaydi — bu xotira kam bo‘lganda ayniqsa qimmatli; pivot tanlash unumdorlikka ta’sir qiladi (tasodifiy bo‘lishi mumkin); divide and conquer, oson parallelizatsiya qilinadi; rekursiv — yomon holatda katta massivda stekni to‘ldirib yuborishi mumkin.

Ikkilik qidirish — **O(log n)** o‘rtacha va yomon holatda: ro‘yxat tartiblangan bo‘lishi kerak; hozirgi qismning o‘rtacha elementi bilan solishtirish — yangi qism ikki barobar kichik bo‘lishi kafolatlangan (divide and conquer!). Massiv xotirasiga **tasodifiy kirish** talab qilinadi: ketma-ket ma’lumotlar (qattiq disk) da oldinga-orqaga qidirish xarajatli — ba’zan chiziqli qidirish yaxshiroq. stdlib.h da bsearch() sifatida amalga oshirilgan:

```
int * binary_search(int val) {
    unsigned int L = 0, R = array_length(arr), M;
    while (L < R) {
        M = (L + R - 1) / 2;
        if (val == arr[M])
            return arr + M;  /* found */
        else if (val < arr[M])
            R = M;           /* in first half */
        else
```

```
            L = M + 1;       /* in second half */
    }
    return NULL; /* not found */
}
```

[Manba: Lecture$_{71}$, slaydlar 21–32]

Rekursiya: asosiy holat, kichikroq masala, progress

Rekursiya — bu funksiya kichikroq masala uchun o‘zini o‘zi chaqiradi. "Navbat" aynanligi: navbatdagi sizgacha necha kishi bor, sizni hisobga olgan holda? Keyingisiga so‘rang va kuting; oxirgi "faqat men — 1" deb javob beradi; javoblar qaytadi, har biri o‘zini qo‘shadi: 1 + 1 = 2, 2 + 1 = 3, 3 + 1 = 4. Xususiy usul, kichikroq masala, to‘g‘ridan-to‘g‘ri hal qilinadigan holatgacha.

Funksiya chaqiruvi **chaqiruvchini to‘xtatadi**: main double$_{value}$(3) ni chaqiradi va kutadi; chaqirilgan funksiyada o‘z x = 3; u 6 ni qaytaradi; main result = 6 bilan davom etadi. (6 ni qaytarish va 6 ni chop etish — turli narsalar.)

```
int double_value(int x)
{
    return 2 * x;
}
// Inside main:
int result = double_value(3);
```

Birinchi rekursiv funksiya — teskay sanash:

```
void count_down(int n)
{
    if (n == 0)
        return;
    printf("%i\n", n);
    count_down(n - 1);
}
```

Asosiy holat: 0 da — bu chaqiruvni to‘xtatish. **Rekursiv chaqiruv**: bir xil funksiya, kichroq kirish. count$_{down}$(3) trassirovkasi: 3, 2, 1 ni chop etadi (har bir son alohida satrda); count$_{down}$(0) chop etmasdan qaytadi — 0 ga yetiladi, lekin chop etilmaydi.

Rekursiyaning uchta tarkibiy qismi: (1) asosiy holat — yangi chaqiruvsiz javob (n == 0); (2) kichikroq masala - soddaroq kirishdagi bir xil funksiya (count$_{down}$(n - 1)); (3) progress — har bir bosqim asosiy holatga yaqinlashadi (3, 2, 1, 0).

Rekursiv chaqiruvdan oldinmi yoki keyinmi? Chaqiruvdan oldin chop etish - qiymatlar pasayishda paydo bo‘ladi; chaqiruvdan keyin chop etish - chop etish kichikroq chaqiruvning qaytishini kutadi:

```
void count_down(int n)
{
    if (n == 0)
        return;
    printf("%i\n", n);
    count_down(n - 1);
}

void count_up(int n)
{
    if (n == 0)
        return;
    count_up(n - 1);
    printf("%i\n", n);
}
```

	$count_{down}(3)$	$count_{up}(3)$
Chiqarish	3 2 1	1 2 3
Chop etish	chuqurlashishdan oldin — pasayishda	kichikroq chaqiruvning qaytishini kutadi

Chop etish tartibi bajarish tartibiga mos keladi [Manba: Recursion-Lecture, slaydlar 14–15]

Asosiy holatga erishish kerak: kodda asosiy holat bo‘lishi dasturni to‘xtatishni ta’minlamaydi. Xuddu shu n bilan rekursiv chaqiruv 3, 3, 3, 3… chiqaradi — kirish hech qachon 0 ga yetmaydi; tuzatish — $count_{down}(n - 1)$. Bu oddiy funksiya uchun manfiy kirish uni buzib yubolar edi — kirish shartining (n >= 0) ahamiyati muhim; cheksiz rekursiya stekni to‘ldirib yuborishi mumkin:

```
void count_down(int n)
{
    if (n == 0)
        return;
    printf("%i\n", n);
    count_down(n - 1);   /* 3, 3, 3, 3, ... — kirish 0 ga yetmaydi */
}

/* Tuzatish */
count_down(n - 1);
```

$sum_{to}(n)$ — 1 dan n gacha butun sonlar yig‘indisini ($sum_{to}(0) = 0$) ko‘chirish-tekshirish: bo‘sh joyda n - 1 bo‘lishi kerak; $sum_{to}(3) = 3 + 2 + 1 + 0 = 6$; n = 3 bo‘lgan kadrdagi "kichik javobga 3 qo‘shish" ishini kutmoqda:

```
int sum_to(int n)
{
    if (n == 0)
        return 0;
    return n + sum_to(n - 1);
}
```

[Manba: Recursion-Lecture, slaydlar 2–8, 14–17]

Faktorial, chaqiruvlar stacki, fibonachchi va Xanoy

Faktorial: 3! = 3 × 2 × 1 = 6; 2 × 1 qismi 2! ga teng, shuning uchun 3! = 3 × 2! — "ichki kichik javob". Asosiy holat: **0! = 1** (agar asos 0 qaytarsa, har bir ko‘paytma 0 bo‘lib qolardi):

```
int factorial(int n)
{
    if (n == 0)
        return 1;
    int smaller = factorial(n - 1);
    return n * smaller;
}
```

Har bir chaqiruvning o‘z stack kadri bor: kadrlar (stack frame, activation record) tugallanmagan chaqiruv uchun kerak bo‘lgan narsalarni saqlaydi: o‘z parametrlari/lokal o‘zgaruvchilar va davom etish nuqtasini. factorial(3) kadri: n = 3; factorial(2) ni kutmoqda; keyingi harakat — return 3 * smaller. Yangi chaqiruv o‘z n ni oladi; chaqiruvchi o‘zining tugallanmagan ishini saqlaydi.

factorial(3) ni trassirovka qilish: main natijani kutmoqda; factorial(3) factorial(2) ni chaqiradi va 3 ga ko‘paytirishni kutmoqda; factorial(2) → factorial(1) → factorial(0). **Faqat yuqori chaqiruv faol.** factorial(0) asosiy holati 1 qaytaradi — lekin bu yakuniy natija emas: asosiy holatga erishish 6 ni main ga qaytirmaydi. Qaytishlar tugallangan kadrlarni olib tashlaydi: factorial(1) smaller = 1 ni oladi, 1 × 1 = 1 ni hisoblaydi; factorial(2): 2 × 1 = 2; factorial(3): 3 × 2 = 6; main result = 6 bilan davom etadi. Chaqiruvlar tartibi: 3, 2, 1, 0; **qaytishlar: 1, 1, 2, 6**.

	Rekursiv faktorial	Tsiklik faktorial
Takroriy qadam	funksiya o‘zini chaqiradi	tsikl tanasini takrorlaydi
Nima kutmoqda	har bir darajada bitta tugallanmagan chaqiruv	rekursiv chaqiruvlar yo‘q
Bu misoldagi holat	har bir chaqiruv o‘z n ni saqlaydi	hisoblagich va to‘planayotgan ko‘paytma
3 uchun natija	6	6

Rekursiya va tsikl [Manba: Recursion-Lecture, slayd 18]

Rekursiyani har doim tezroq yoki yaxshiroq deb da’vo qilmang: faktorial uchun tsikl va rekursiya bir xil natija beradi va ko‘paytirishlar soni o‘xshash; lekin o‘ziga o‘xshash masalalar (ichki joylashgan tuzilmalarni o‘tkazib o‘tish) uchun rekursiv shakl odatda soddaroq ifodalanadi.

Fibonachchi: 0, 1, 1, 2, 3, 5, 8, … — har keyingi qiymat ikki oldingi yig‘indisiga teng. **Ikkita asosiy holat** (birinchi ikki qiymat to‘g‘ridan-to‘g‘ri ma’lum: F(0) = 0, F(1) = 1); ikkita kichik javob — avval chap chaqiruv to‘liq tugaydi, keyin o‘ng boshlanadi, keyin ular qo‘shiladi:

```
int fibonacci(int n)
{
    if (n == 0)
        return 0;
    if (n == 1)
        return 1;
    int left = fibonacci(n - 1);
    int right = fibonacci(n - 2);
    return left + right;
}
```

fibonacci(4) 3 qaytaradi. fibonacci(4) orqasida to‘qqizta chaqiruv (koddagi aniq tartibda chapdan o‘ngga o‘qiladi: 4, 3, 2, 1, 0, 1, 2, 1, 0): fibonacci(3) → 2; fibonacci(2) → 1 (ikki marta!); fibonacci(1) → 1 (uch marta); fibonacci(0) → 0 (ikki marta). **fibonacci(2) ikki marta uchraydi — bir xil javob qayta hisoblanadi**; har bir vaqt faqat bitta chaqiruv bajariladi; bu oddiy versiya ishni takrorlaydi.

- fibonacci(4) → 3 qaytaradi
- fibonacci(3) → 2 qaytaradi
- fibonacci(2) → 1 qaytaradi (va yana 1 — ikkinchi tugun)
- fibonacci(1) → 1 qaytaradi (uchta chaqiruv)
- fibonacci(0) → 0 qaytaradi (ikki chaqiruv)

Xanoy minorasi (Book Exercise 8.5): barcha uch diskni A dan C ga, B dan foydalanib ko‘chirish. Qoidalar: (1) bir vaqtda faqat bitta diskni ko‘chirish; (2) faqat ustun diskni ko‘chirish; (3) hech qachon katta diskni kichik ustiga qo‘yish. Uchta rekursiv qadam: (1) 2 diskni A dan B ga, C dan foydalanib ko‘chirish — 3 bir diskli harakat; (2) 3 diskni A dan C ga ko‘chirish — 1 harakat; (3) 2 diskni B dan C ga, A dan foydalanib ko‘chirish — 3 harakat. Jami **3 + 1 + 3 = 7 harakat**. Asosiy holat: disk yo‘q — harakat yo‘q (n == 0). Yetti harakatning aniq ketma-ketligi: 1 disk A→C; 2 disk A→B; 1 disk C→B; 3 disk A→C; 1 disk B→A; 2 disk B→C; 1 disk A→C.

[Manba: Recursion-Lecture, slaydlar 7–13, 19–22]

5.2. 7-hafta savollari — ma’ruza va laboratoriya savollari model javoblar bilan

S1. Nima bu ko‘rsatkich va nima bu nolli ko‘rsatkich?

Javob. Ko‘rsatkich — bu o‘zgaruvchining xotiradagi manzili; manzil o‘zgaruvchiga istalg joydan murojaat qilish va uni o‘zgartirish imkonini beradi (ma’lumotlar tuzilmalari uchun ayniqsa foydalali). Nolli ko‘rsatkich — 0 (NULL): hech narsaga ishlamaydigan ko‘rsatkich.

S2. & va * operatorlari nima qiladi?

Javob. & — manzil olish operatori: t turidagi o‘zgaruvchining manzili t *turiga ega (int* pn = &n;). — *ishga tushirish/ko‘ndalanglik operatori: manzil bo‘yicha qiymatga kirish (printf("pi = %g\n",* ppi); yoki *ppi* = ppi + *pn;). Ishga tushirilgan ko‘rsatkich oddiy o‘zgaruvchidek harakat qiladi.

S3. Funksiyadan bir nechta qiymatni qanday qaytarish mumkin?

Javob. Qo‘shimchi chiqarishlar uchun ko‘rsatgichlarni uzatish: int ext_{euclid}(int a, int b, int *x, int* y); chaqirish g = ext_{euclid}(a, b, &x, &y); x va y ni uzatilgan manzillarga yozadi. Funksiyalar bittadan ortiq qiymat qaytarmaydi — ko‘rsatgichlar bu chekni kengaytiradi.

S4. Lokal massiv manzilini qaytaruvchi $get_{message}$() funksiyasida nimadir noto‘g‘ri?

Javob. char msg[] $get_{message}$() funksiyasining stek kadrida yashaydi; funksiya qaytgandan so‘ng kadr yo‘qoladi va ko‘rsatgich yaroqsiz (dangling) bo‘lib qoladi — puts(string) yo‘qolgan xotirani o‘qiydi. Kompylerlar odatda ogohlantiradi. Yechimlar: statik-massiv, chaqiruvchi tomonidan uzatilbufyer yoki malloc-xotir, uni chaqiruvchi bo‘shatadi.

S5. Nima uchun swap(int a, int b) main dagi x va y ni o‘zgartirmaydi va qanday tuzatish mumkin?

Javob. C qiymat nusxalarini uzatadi: swap o‘zining lokal nusxalarini o‘zgartiradi, main o‘zgaruvchulari tegilmaydi. Tuzatish — ko‘rsatgichlar: void swap(int *x, int* y) { int temp = *x;* x = *y;* y = temp; } chaqirish swap(&a, &b); undan keyin a = 7, b = 5.

S6. pa + i nima beradi va bu oddiy sonlarni qo‘shish emasligi nega?

Javob. pa + i arr[i] ni ko‘rsatadi: manzil ma’lumotlar turining o‘lchami bo‘yicha i ga ko‘paytirilgacha oshadi. Ko‘rsatgich — int emas: arifmetika element o‘lchami bilan masshtablanadi (int uchun 64-bitli platformada 4 bayt qadam).

S7. Agar arr[0] manzili 100 bo‘lsa (int massiv), arr[3] ning manzili qancha? Va char *pc = (char*)pa bo‘lsa — qaysi i (int *)(pc + i) == pa + 3 ni beradi?

Javob. arr[3] manzili 112: 100 + 3 × 4 bayt (int o‘lchami). Char-ko‘rsatgich uchun qadam 1 bayt, shuning uchun i = 12: 3 int = 12 bayt.

S8. string.h da qanday sat ishlash funksiyalari mavjud?

Javob. Nusxalash: strcpy(strto, strfrom), strncpy(strto, strfrom, n). Solishtirish: strcmp(str1, str2) — 0 teng bo‘lganda, >0 agar str1 > str2, <0 agar str1 < str2; strncmp — birinchi n belgisi. Uzunlik: strlen(str). Konkatenatsiya: strcat, strncat. Belgini qidirish: strchr — birinchi uchrash, strrchr — oxirgi (NULL, agar topilmasa).

S9. 7-maqoladagi to‘rtta algoritmning murakkabligini noming.

Javob. Insertion sort — $O(n^2)$. Quicksort — O(n log n) o‘rtacha, $O(n^2)$ yomon holatda (stdlib.h da qsort() sifatida amalga oshirilgan). Chiziqli qidirish — o‘rtacha yarim elementni tashrif buyuradi. Ikkilik qidirish — O(log n) o‘rtacha va yomon holatda (tartirilgan ma’lumotlarni va tasodifiy kirishni talab qiladi; bsearch() sifatida amalga oshirilgan).

S10. Rekursiyaning uchta ingredienti nima?

Javob. (1) Asosiy holat — boshqa chaqiruvsiz javob (n == 0). (2) Kichikroq masala — bir xil funksiya soddaroq kirishda ($count_{down}$(n - 1)). (3) Progress — har qanday qabul qilinadigan rekursiv yo‘l asosiy holatga yaqinlashadi (3, 2, 1, 0).

S11. $count_{down}$(3) va $count_{up}$(3) nima chiqaradi va nega chiqarish farq qiladi?

Javob. $count_{down}$(3) 3 2 1 chiqaradi, $count_{up}$(3) — 1 2 3. $count_{down}$ da chiqarish rekursiv chaqiruvdan oldin turadi — qiymatlar tushishda paydo bo‘ladi. $count_{up}$ da chiqarish chaqiruvdan keyin turadi — u kichikroq chaqiruvning qaytishini kutadi va qaytishda bajariladi. Ikkala funksiya ham n = 0 ga tashrif buyuradi va to‘xtaydi.

S12. Nima uchun $count_{down}$ rekursiv chaqiruv $count_{down}$(n) bilan aylanib chiqadi va qanday tuzatish mumkin?

Javob. Kirish hech qachon 0 ga yetmaydi: har bir chaqiruv bir xil n ni oladi, 3, 3, 3, 3… chiqariladi — kodda asosiy holat mavjudligi yetarli emas, har bir yo‘l progress qilishi kerak. Tuzatish: $count_{down}$(n - 1). Cheksiz rekursiya stek bo‘shligini ishlatib bo‘lishi mumkin.

S13. fibonacci(4) qaytaradi va fibonacci(2) ikki marta uchraydi?

Javob. fibonacci(4) 3 ni qaytaradi. Chaqiruvlar daraxti to‘qqizta chaqiruvni o‘z ichiga oladi: 4, 3, 2, 1, 0, 1, 2, 1, 0 — ikkala fibonacci(2) pastki daraxtlari bir xil javobni mustaqil hisoblaydi: ikkinchi shox birinchisini eslamaydi. Faqatgina bitta chaqiruv har bir vaqtda bajariladi; oddiy versiya ishni takrorlaydi.

S14. Xanoy minorasi: uchta rekursiv bosqichni va uchta disk uchun umumiy harakatlar sonini noming.

Javob. (1) 2 diskni A dan C yordamida B ga ko‘chirish — 3 harakat; (2) 3-diskni A dan C ga ko‘chirish — 1 harakat; (3) 2 diskni B dan A yordamida C ga ko‘chirish — 3 harakat. Jami 3 + 1 + 3 = 7 harakat. Asosiy holat: disklar yo‘q — harakatlar yo‘q (n == 0). Qoidalar: bir disk bir vaqtda, faqat yuqori disk, hech qachon katta disk kichik ustida.

5.3. 7-hafta testlari

T1. & operatori o‘zgaruvchiga qo‘llanilgida nima qiladi?

a) o‘zgaruvchi manzilini qaytaradi
b) ko‘rsatkichni bekor qiladi
c) o‘zgaruvchini o‘ziga ko‘paytiradi
d) ko‘rsatkichni e’lon qiladi

Javob: a) o‘zgaruvchi manzilini qaytaradi

*Tushuntirish: "O‘zgaruvchi manzilini qanday topish? & operatori": int *pn = &n; — butun son n ning manzili.*

T2. Agar int n; bo‘lsa, &n ifodasi qaysi turda bo‘ladi?

a) int
b) int *
c) double *
d) void

Javob: b) int *

Tushuntirish: "t turidagi o‘zgaruvchi manzili t turiga ega" — int manzili int turiga ega.

T3. *ppi* = ppi + *pn; nima qiladi?

a) ko‘rsatuvchi orqali o‘zgaruvchi qiymatini o‘zgartiradi
b) ppi manzilini o‘zgartiradi
c) kompilyatorda xatoga olib keladi
d) hech narsa qilmaydi

Javob: a) ko‘rsatuvchi orqali o‘zgaruvchi qiymatini o‘zgartiradi

Tushuntirish: chapda — manzilga yozish (pi endi 7.14159), o‘ngda — bekor qilish orqali qiymatlarni o‘qish; bekor qilingan ko‘rsatuvchi oddiy o‘zgaruvchi kabi harakat qiladi.

T4. Quyidagilardan qaysi biri manzilga ega emas?

a) xotiradagi o‘zgaruvchi
b) register-o‘zgaruvchi
c) massiv elementi
d) ko‘rsatuvchi

Javob: b) register-o‘zgaruvchi

Tushuntirish: "Qaysi biri manzilga ega emas? register o‘zgaruvchilari; doimiy/so‘zlar/preprotsessor aniqlanishi; ifodalar (agar natija o‘zgaruvchi bo‘lmasa)".

T5. 64-bitli platformada int arr[8]; uchun sizeof(arr) qanchaga teng?

a) 8
b) 32
c) 4
d) 64

Javob: b) 32

Tushuntirish: primitiv massivlar uchun sizeof massiv hajmini baytlarda beradi: 8 element × 4 bayt = 32; long arr[5] 40 beradi.

T6. E’lon qilingan o‘zgaruvchilar qayerga, dinamik xotira esa qayerga tushadi?

a) ikkalasi ham stakka
b) ikkalasi ham xotiraga
c) e’lon qilinganlar — stakka, dinamik — xotiraga
d) e’lon qilinganlar — xotiraga, dinamik — stakka

Javob: c) e’lon qilinganlar — stakka, dinamik — xotiraga

Tushuntirish: "Virtual xotira foydali qismlari: stak va xotira. stak: e’lon qilingan o‘zgaruvchilar keladigan joy; xotira: dinamik xotira keladigan joy".

T7. strcmp(str1, str2) qachon 0 qaytaradi?

a) str1 str2 dan katta
b) satrlar teng
c) str1 str2 dan kichik
d) satrlar har xil uzunlikda

Javob: b) satrlar teng

Tushuntirish: strcmp agar teng bo‘lsa 0 qaytaradi; agar str1 > str2 bo‘lsa musbat; agar str1 < str2 bo‘lsa manfiy.

T8. strchr(str, c) nima qiladi?

a) satrda kichik satr qidiradi
b) belgi c ning birinchi uchrashiga ko‘rsatuvchini yoki NULL qaytaradi
c) c ning oxirgi uchrashishini qaytaradi
d) satr uzunligini hisoblaydi

Javob: b) belgi c ning birinchi uchrashiga ko‘rsatuvchini yoki NULL qaytaradi

Tushuntirish: strchr — birinchi uchrashish, strrchr — oxirgi; ikkalasi ham agar belgi topilmasa NULL qaytaradi.

T9. Rekursiyaning asosiy holati bu…

a) yangi rekursiv chaqiruvsiz olingan javob
b) funksiyaning birinchi chaqiruvi
c) funksiya ichidagi tsikl
d) main dan oxirgi qaytish

Javob: a) yangi rekursiv chaqiruvsiz olingan javob

Tushuntirish: "Asosiy holat: boshqa chaqiruvsiz javob" — $count_{down}$ da bu n == 0: return;. Boshqa ingredientlar: kichikroq masala va progress.

T10. $count_{down}(3)$ nima chiqaradi?

a) 3 2 1
b) 1 2 3
c) 3 3 3
d) 0 1 2 3

Javob: a) 3 2 1

Tushuntirish: chiqarish rekursiv chaqiruvdan oldin turadi — qiymatlar tushishda paydo bo'ladi; $count_{up}(3)$ chaqiruvdan keyin chiqarish bilan 1 2 3 beradi.

5.4. Atamalar — 7-hafta glossariysi

Atama	Ma'nosi
Ko‘rsatuvchi (pointer)	xotiradagi o‘zgaruvchi manzili; o‘zgaruvchiga istalgan joydan kirish/o‘zgartirish; ma'lumot tuzilmalari asosi
& operatori (address-of)	o‘zgaruvchi manzilini qaytaradi; t turidagi o‘zgaruvchi manzili t * turiga ega
Bekor qilish (*) (dereferencing/indirection)	manzil bo‘yicha qiymatni o‘qish/yozish: *ppi* = ppi + *pn;
Nol ko‘rsatuvchi (NULL)	hech narsaga ishlamaydigan 0 ko‘rsatuvchi
Virtual xotira	OS abstraksiyasi: kodga kirish mumkin bo‘lgan manzil maydoni; fizik xotiraga xaritalanadi
Stak (stack)	virtual xotira qismi, bu yerga e'lon qilingan o‘zgaruvchilar (va chaqiruv kadrlari) tushadi
Xotira (heap)	bu yerga dinamik xotira tushadi
Osilib qolgan ko‘rsatuvchi (dangling)	ko‘lamdan chiqib ketgan o‘zgaruvchiga ko‘rsatuvchi (mahalliy massiv manzilini qaytarish)
Manzil arifmetikasi	pa + i arr[i] ni ko‘rsatadi; manzil i × tur o‘lchami oshadi; arr[0] 100 manzilda → arr[3] 112 da
Nol-terminal ('\0') va strlen	C-satrini tugatadigan nol bayti; strlen(str) terminalsiz uzunlikni qaytaradi — nusxalash uchun +1 bayt zaxira kerak
Asosiy holat (base case)	yangi rekursiv chaqiruvsiz javob: n == 0 → return 1 faktorialda; Fibonachchining ikkita asosiy holati
Chaqiruvlar stacki / kadrlar (stack frame, activation record)	tugallanmagan chaqiruv yozuvi: o‘z parametrlari, lokal o‘zgaruvchilar, davom etish nuqtasi
O-notatsiyasi (Big-O)	algoritmning o‘sish tartibi: insertion $O(n^2)$; quicksort O(n log n) o‘rtacha holatda, $O(n^2)$ yomon holatda; binary search O(log n)
Xanoy minorasi (Tower of Hanoi)	klassik rekursiv masala: 3 diskni A dan C ga B orqali 3 + 1 + 3 = 7 harakatda; asosiy holat — disklar yo‘q, harakatlar yo‘q

6. Bug Hunts — barcha xatolar va ularning tuzatilishi

Kursning uchta ish jadvali — "xato ovini": **bug$_{huntc}$.txt** (shartli konstruktsiyalar va funksiyalar), **bug$_{huntloops}$.txt** (tsikllar) va **bug$_{huntpointersstrings}$.txt** (ko‘rsatkichlar va satrlar, o‘qituvchi kaliti bug$_{huntpointersstringsanswerkey}$.txt). Jadvaldagi o‘yin qoidalari: juftlikda ishlash; har bir dasturda xatoni topib, nima uchun noto‘g‘ri ekanini tushuntirish va tuzatish; taklif qilingan kirishlar bilan kompilyatsiya qilish va sinash. Kalit maslahati: gcc -g -Wall -fsanitize=address bilan kompilyatsiya qiling va tuzatmalarni tekshiring — sanitizer xabarlari muhokama uchun ajoyib mavzu. Quyida har bir masala to‘liq tahlil qilingan: harfma-harf ro‘yxat, tushuntirish va tuzatilgan kod.

6.1. Bug Hunt C

"BUG HUNT: CONDITIONALS AND FUNCTIONS (C)" jadvali — oltita masala. Tez raund (10 daqiqa): masalalar 1, 2, 5; muhokama qilinadigan masalalar: 3 va 4 (xatolar faqat ma’lum kirishlarda ko‘rinadi); C-ga xos darslar: 2, 5 va 6 (kompilyator xatolari va inicializatsiyalanmagan o‘zgaruvchilar). O‘qituvchi kaliti faylga kiritilgan va quyida keltirilgan.

B1. Baholovchi — shartlar tartibi

Kutilayotgan: 95 → 'A', 85 → 'B', 75 → 'C', 65 → 'D', 40 → 'F'. Xatoli ro‘yxat (harfma-harf):

```
#include

char grade(int score) {
     if (score >= 90) {
        return 'A';
     else if (score >= 80) {
        return 'D';
    } else if (score >= 70) {
        return 'C';
    } else if (score >= 60) {
        return 'B';
    } else
    } else {
        return 'F';
    }
}

int main(void) {
char student1 = grade(95);
char student2 = grade(78
    printf("%c\n", grade(95));
    puts("hello");
    return 0;
}
```

B1. grade dasturida nima noto‘g‘ri va uni qanday tuzatish kerak?

Javob. Kalit bo‘yicha: shartlar noto‘g‘ri tartibda tekshiriladi — birinchi eng yuqori chegarasi (>= 90, keyin >= 80, keyin >= 70, keyin >= 60); noto‘g‘ri tartib tufayli score = 95 birinchi navbatda (score >= 60) shoxchasiga tushadi va 'D' qaytaradi, qolgan shoxchalar erishib bo‘lmaydi. Bosma ro‘yxatda tartib harflar bilan chalkashib ketgan: (>= 80 shoxchasi 'D' qaytaradi, >= 60 esa 'B') va sintaksis buzilgan: return 'A'; dan keyin yopuvchi qavs yo‘q, else takrorlangan, grade(78 chaqiruvi yopilmagan — dastur hatto kompilyatsiya qilinmaydi. Tuzatish: chegaralarni kamayish tartibida tekshirish, har bir shoxchaga o‘z harfini qaytarish va qavslarni tiklash:

```
#include
```

```
char grade(int score) {
    if (score >= 90) {
        return 'A';
    } else if (score >= 80) {
        return 'B';
    } else if (score >= 70) {
        return 'C';
    } else if (score >= 60) {
        return 'D';
    } else {
        return 'F';
    }
}

int main(void) {
    char student1 = grade(95);
    char student2 = grade(78);
    printf("%c\n", grade(95));
    puts("hello");
    return 0;
}
```

[Manba: bug_{huntc}.txt, MASALA 1 + kalit]

B2. Juft yoki toq — solishtirish va yo‘qolgan return

Kutilayotgan: $is_{even}(4) \to 1$, $is_{even}(7) \to 0$. Xatoli ro‘yxat (harfma-harf):

```
#include

int is_even(int n) {
    int result;
    if (n % 2 == 0) {
        result = 0;
    } else {
        result = 1;
    }
    return result;
}


int main(void) {
result = 5;
    printf("%d\n", is_even(4));
    return 0;
}
```

B2. is_{even} dasturida nima noto‘g‘ri va uni qanday tuzatish kerak?

Javob. Kalit ikki xatoni ko‘rsatadi: (a) "n % 2 = 0" — taqqoslash o‘rniga tayinlash, == kerak (GCC xato haqida xabar beradi: ifodaga tayinlab bo‘lmaydi); (b) funksiya result ni qaytarmaydi — return result yo‘q; xulq-atoma belgilanmagan. Bosma ro‘yxatda solishtirish allaqachon == ga tuzatilgan, lekin shoxchalar o‘rniga almashtirilgan: juft son 0 ga, toq son 1 ga tayinlanadi, main result funksiya tashqarisida chaqiradi — main da o‘zgaruvchi result mavjud emas. To‘g‘ri: juft → 1, toq → 0, result ni qaytarish, result = 5; qatorini olib tashlash main dan:

```
#include

int is_even(int n) {
    int result;
```

```
    if (n % 2 == 0) {
        result = 1;
    } else {
        result = 0;
    }
    return result;
}

int main(void) {
    printf("%d\n", is_even(4));
    return 0;
}
```

[Manba: bug_{huntc}.txt, MASALA 2 + kalit]

B3. Kabisa yili — erishib bo‘lmaydigan % 400 shoxchasi

Kutilayotgan: 1900 → 0, 2000 → 1, 2024 → 1, 2023 → 0. Ro‘yxat (harfma-harf):

```
#include

int is_leap(int year) {
    if (year % 4 == 0) {
        return 1;
    } else if (year % 100 == 0) {
        return 0;
    } else if (year % 400 == 0) {
        return 1;
    } else {
        return 0;
    }
}

int main(void) {
    printf("%d\n", is_leap(1900));
    printf("%d\n", is_leap(2000));
    return 0;
}
```

B3. is_{leap}(1900) nima uchun 1 qaytaradi, garchi 1900 oddiy yil?

Javob. Har qanday 4 ga karrali yil darhol 1 qaytaradi, shuning uchun 1900 noto‘g‘ri ravishda kabisa yili deb hisoblanadi va % 400 shoxchasi hech qachon bajarilmaydi — u % 4 dan keyin keladi. Kalit buyuradi: % 400 birinchi tekshirish, keyin % 100, keyin % 4 — yoki ixcham ifodadan foydalanish (4-hafta laboratoriyasidagi shart bilan bir xil): return (year % 4 == 0 && year % 100 != 0) || year % 400 == 0;.

```
#include

int is_leap(int year) {
    if (year % 400 == 0) {
        return 1;
    } else if (year % 100 == 0) {
        return 0;
    } else if (year % 4 == 0) {
        return 1;
    } else {
        return 0;
    }
}

/* yoki bir qatorda, 4- hafta kaliti va laboratoriyasi bo'yicha: */
int is_leap(int year) {
```

```
    return (year % 4 == 0 && year % 100 != 0) || year % 400 == 0;
}
```

[Manba: bug_{huntc}.txt, MASALA 3 + kalit]

B4. Uchta orasidagi eng katta — eng kattalarning tengligi

Kutilayotgan: biggest(5, 5, 3) → 5. Ro‘yxat (harfma-harf):

```
#include

int biggest(int a, int b, int c) {
    if (a > b && a > c) {
        return a;
    } else if (b > a && b > c) {
        return b;
    } else {
        return c;
    }
}

int main(void) {
    printf("%d\n", biggest(5, 5, 3));
    printf("hello");
    puts("hello")
    return 0;
}
```

B4. biggest(5, 5, 3) nima uchun 3 qaytaradi?

Javob. Eng katta qiymatlar tengligi (5 va 5) holatida hech qanday qat’iy taqqoslanish ">" to‘g‘ri emas: birinchi ikkita shart noto‘g‘ri, bajarish else qismiga o‘tadi va c = 3 ni qaytaradi. Kalit: taqqoslashda >= dan foydalaning. Bundan tashqari, main da puts("hello") dan keyin nuqta-vergul qo‘yib yuborilgan — bu sintaktik xato, uni ham tuzatish kerak:

```
#include

int biggest(int a, int b, int c) {
    if (a >= b && a >= c) {
        return a;
    } else if (b >= a && b >= c) {
        return b;
    } else {
        return c;
    }
}

int main(void) {
    printf("%d\n", biggest(5, 5, 3));
    printf("hello");
    puts("hello");
    return 0;
}
```

[Manba: bug_{huntc}.txt, PROBLEM 4 + kalit]

B5. Double and add — ifodadagi void-funksiya

Kutilgan: total = 16 (10 + 6). Listing (so‘zma-so‘z):

```
#include

void doubled(int x) {
```

```
    printf("%d\n", x * 2);
}

int main(void) {
    int total = doubled(5) + doubled(3);
    printf("%d\n", total);
    return 0;
}
```

B5. Nima uchun int total = doubled(5) + doubled(3); kompilyatsiyalanmaydi?

Javob. doubled void sifatida e’lon qilingan — u hech narsa qaytarmaydi va ifodada foydalanib bo‘lmaydi; u qaytarish o‘rniga chop etadi. Kompilyator xabar beradi: "void value not ignored as it ought to be". Kalit: qaytarish turini int ga o‘zgartiring va return x * 2; dan foydalaning, chop etishni main ga o‘tkazing. Keyin total = 10 + 6 = 16, kutilgandek:

```
#include

int doubled(int x) {
    return x * 2;
}

int main(void) {
    int total = doubled(5) + doubled(3);
    printf("%d\n", total);
    return 0;
}
```

[Manba: bug_{huntc}.txt, PROBLEM 5 + kalit]

B6. Member discount — inicializatsiyalanmagan o‘zgaruvchi

Kutilgan: $apply_{discount}(100, 0) \rightarrow 100.0$, $apply_{discount}(100, 1) \rightarrow 80.0$. Listing (so‘zma-so‘z):

```
#include

double apply_discount(double price, int is_member) {
    double discount;
    if (is_member) {
        discount = 0.2;
    }
    return price * (1 - discount);
}

int main(void) {
    printf("%.2f\n", apply_discount(100, 0));
    return 0;
}
```

B6. is_{member} = 0 bo‘lganda discount qanday qiymatga ega va bu nima xavfli?

Javob. is_{member} = 0 bo‘lganda if sharti bajarilmaydi va discount inicializatsiyalanmagan qoladi — natija chindan ham chiqindilar (undefined behavior) bo‘ladi. Bu C-ga xos tuzoq: C da avtomatik standart qiymat yo‘q. Kalit: o‘zgaruvchini inicializatsiya qiling (double discount = 0.0;) yoki else qo‘shing:

```
#include

double apply_discount(double price, int is_member) {
    double discount = 0.0;
    if (is_member) {
        discount = 0.2;
    }
```

```
    return price * (1 - discount);
}

int main(void) {
    printf("%.2f\n", apply_discount(100, 0));
    return 0;
}
```

[Manba: bug_{huntc}.txt, PROBLEM 6 + kalit]

6.2. Bug Hunt Loops

List "C++ BUG HUNT: LOOPS" deb nomlangan va #include <iostream>, <vector> va using namespace std; fragmentlarini nazarda tutadi, listingda printf va cout dan foydalanilgan — listing so‘zma-so‘z saqlangan. List qoidalari: har bir dastur aynan BIR, looplar bilan bog‘liq xatoga ega; kutilgan xulq-atvorni o‘qing; xatoni topib chizing; bitta gapda tushuntiring; tuzatishni yozing. Faylda rasmiy kalit yo‘q — pastdagi tahlillar har bir fragmentning kutilgan xulq-atvoriga ko‘ra tuzilgan.

L1. Baholar yig‘indisi — tekshiruv masalasi

Kutilgan: "Sum: 405" chop etadi. Listing (so‘zma-so‘z):

```
    int scores[5] = {90, 85, 70, 100, 60};
    int sum = 0;
    for (int i = 0; i < 5; i++) {
        sum += scores[i];
    }
    printf("Sum: %d", sum);
```

L1. Bu loopdagi xatoni toping.

Javob. Bu tekshiruv masalasi: chop etilganidek, loop to‘g‘ri — i 0-4 indekslarni o‘tadi, har bir element bir marta qo‘shiladi va yig‘indi 90 + 85 + 70 + 100 + 60 = 405, shuning uchun dastur aynan "Sum: 405" ni chop etadi, kutilgandek. Saboq: ishlaydigan kodni "tuzatish"dan oldi, qo‘lda natijani hisoblang — har bir loopli fragmentda xato bo‘lishi shart emas (yagona rasmiy noaniqlik — da printf da \n satir o‘tkazgich yo‘q, bu natijaga ta’sir qilmaydi). Tuzatish kerak emas:

```
int scores[5] = {90, 85, 70, 100, 60};
    int sum = 0;
    for (int i = 0; i < 5; i++) {
        sum += scores[i];
    }
    printf("Sum: %d\n", sum); /* \n qo'shilgan — ixtiyoriy */
```

[Manba: $bug_{huntloops}$.txt, BUG 1]

L2. dan keyin nuqta-vergul

Kutilgan: 1..10 yig‘indisini, ya’ni 55 ni chop etadi. Listing (so‘zma-so‘z):

```
    int sum = 0;
    int i;
    for (i = 1; i <= 10; i++); {
        sum += i;
  }
    printf("%d", sum);
```

L2. Nima uchun 55 o‘rniga 11 chop etiladi?

Javob. for sarlavhasidan keyin darhol kelgan nuqta-vergil loopning jismini bo‘sh operator qiladi: loop o‘n marta "bo‘sh aylanadi", i ni 11 gacha yetkazadi va faqatgina keyin { sum += i; } bloki bir marta i = 11 bilan bajariladi

— sum = 11. Tuzatish: for (i = 1; i <= 10; i++) dan keyin ';' ni olib tashlang:

```
int sum = 0;
int i;
for (i = 1; i <= 10; i++) {
    sum += i;
}
printf("%d", sum);
```

[Manba: $bug_{huntloops}$.txt, BUG 2]

L3. Yo‘qotilgan ochilish qavs

Kutilgan: "Count: 0" … "Count: 4" ni chop etadi va to‘xtaydi. Listing (so‘zma-so‘z):

```
for (int count = 0; count < 5; ++count)
    printf("Count: %d", count);
}
```

L3. Bu dasturni Count: 0 … 4 ni chop etishiga qanday to‘sqinlik qiladi?

Javob. for sarlavhasidan keyin ochilish '{' qavsi yo‘q, shuning uchun loopning jismi faqat printf — bu o‘z-o‘zidan to‘g‘ri chiqish beradi, lekin yakunidagi yolg‘iz '}' sintaktik xato: dastur kompilyatsiyalanmaydi. Tuzatish: sarlavhadan keyin '{' qo‘shing (yoki ortiqcha '}' ni olib tashlang):

```
for (int count = 0; count < 5; ++count) {
    printf("Count: %d", count);
}
```

[Manba: $bug_{huntloops}$.txt, BUG 3]

L4. printf argumentsiz

Kutilgan: "1 2 3 4 Found 5!" ni chop etadi va to‘xtaydi. Listing (so‘zma-so‘z):

```
for (int i = 1; i <= 10; i++) {
    if (i == 5) {
        printf("Found 5!");
        break;
    }
    printf("%i ");
}
```

L4. Nima uchun kutilgan sonlar 1 2 3 4 chiqmaydi?

Javob. printf("%i "); da format %i specificatori bor, lekin argument berilmagan — xulq-atv aniqlanmagan (odatda sonlar o‘rniga chindan ham chiqindilar chop etiladi). Tuzatish: i ni o‘tkazing — printf("%i ", i);. Keyin loop 1 2 3 4 ni chop etadi, i == 5 bo‘lganda "Found 5!" va break loopni tugatadi:

```
for (int i = 1; i <= 10; i++) {
    if (i == 5) {
        printf("Found 5!");
        break;
    }
    printf("%i ", i);
}
```

[Manba: $bug_{huntloops}$.txt, BUG 4]

L5. Belgisi bo‘lmagan sanoqchi manfiy bo‘la olmaydi

Kutilgan: "5 4 3 2 1 0 Liftoff!" ni chop etadi va dastur tugaydi. Listing (so‘zma-so‘z):

```
for (unsigned int i = 5; i >= 0; i--) {
    printf("%d ", i);
```

```
}
printf("Liftoff!");
```

L5. Nima uchun "Liftoff!" hech qachon chop etilmaydi?

Javob. i unsigned int sifatida e'lon qilingan — u noldan kichik bo'lishi mumkin emas, shuning uchun i >= 0 sharti har doim to'g'ri: 0 dan keyin i-- operatori hisoblagichni katta songa "o'rab" qo'yadi (4- haftadagi belgisiz arifmetika printsipi: unsigned wraparound), va tsikl cheksiz bo'lib qoladi. Tuzatish: belgili hisoblagich int, uchun i >= 0 sharti 0 dan keyin noto'g'ri bo'lishi uchun:

```
for (int i = 5; i >= 0; i--) {
    printf("%d ", i);
}
printf("Liftoff!");
```

[Manba: $bug_{huntloops}$.txt, BUG 5]

L6. iteratsiya paytida erase (C++)

Kutilgan: barcha juft sonlarni olib tashlash, {1, 3, 5} qoldirish. Listing (so'zma-so'z):

```
vector nums = {1, 2, 3, 4, 5, 6};
for (auto it = nums.begin(); it != nums.end(); ++it) {
    if (*it % 2 == 0) {
        nums.erase(it);
    }
}
for (int n : nums) {
    cout << n << " ";
}
cout << endl;
```

L6. nums.erase(it) dan keyin iterator nima bo'ladi?

Javob. nums.erase(it) iteratorit ni yaroqsiz qiladi (iterator invalidation), va tsikl darhol yaroqsiz iterator ustunda ++it qiladi — belgilanmagan xulq-atvor. Tuzatish: erase qaytargan iterator (u keyingi elementga ishora qiladi) ni tayinlash va faqat o'chirish bo'lmaganda uni oshirish:

```
vector nums = {1, 2, 3, 4, 5, 6};
for (auto it = nums.begin(); it != nums.end(); ) {
    if (*it % 2 == 0) {
        it = nums.erase(it);
    } else {
        ++it;
    }
}
for (int n : nums) {
    cout << n << " ";
}
cout << endl;
```

[Manba: $bug_{huntloops}$.txt, BUG 6]

L7. double ni tenglikka solishtirish

Kutilgan: 0, 0.1, 0.2, … 0.9 gacha chop etadi va to'xtaydi. Listing (so'zma-so'z):

```
for (double x = 0.0; x != 1.0; x += 0.1) {
    printf("%f ", x);
}
```

L7. Tsikl nima uchun 0.9 da to'xtamaydi?

Javob. 0.1 ikkilik haqiqiy arifmetikada aniq ifodalanmaydi; o‘n qo‘shishdan keyin to‘plangan xato aniq 1.0 dan o‘tib ketadi, x != 1.0 sharti to‘g‘ri bo‘lib qoladi va tsikl 0.9 dan oshib ketadi (4- haftadagi bir xil qoida: float/double da == solishtirishi chekli aniqlik tufayli ishonchsiz). Tuzatish: tartib munosabati sharti x < 1.0:

```
for (double x = 0.0; x < 1.0; x += 0.1) {
    printf("%f ", x);
}
```

[Manba: $bug_{huntloops}$.txt, BUG 7]

L8. O‘zgaruvchi total yashirinishi

Kutilgan: "Total: 15" chop etadi. Listing (so‘zma-so‘z):

```
int total = 0;
for (int i = 1; i <= 5; i++) {
    int total = 0;
    total += i;
}
printf("Total: %d", total);
```

L8. Nima uchun "Total: 0" emas, balki 15 chop etiladi?

Javob. Tsikl ichida tashqi totalni yashiruvchi yangi total o‘zgaruvchi e’lon qilingan (shadowing — 5- darsdagi lokal nmax = 25 bilan bir xil mexanizm): har bir iteratsiya yangich ichki totalni yaratadi, unga i qo‘shadi va uni tashlaydi. Tashqi total hech qachon o‘zgarmaydi. Tuzatish: ichki int total = 0; e’lonini olib tashlash — keyin total += i; tashqi o‘zgaruvchini yangilaydi: 1 + 2 + 3 + 4 + 5 = 15:

```
int total = 0;
for (int i = 1; i <= 5; i++) {
    total += i;
}
printf("Total: %d", total);
```

[Manba: $bug_{huntloops}$.txt, BUG 8]

L9. To‘g‘ri o‘zgaruvchini oshirish

Kutilgan: 3×3 ko‘paytirish jadvali: qatorlar 1 2 3 / 2 4 6 / 3 6 9. Listing (so‘zma-so‘z):

```
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; i++) {
        cout << i * j << "\t";
    }
    cout << endl;
}
```

L9. Nima uchun ichki tsikl hech qachon tugamaydi?

Javob. Ichki tsikl j o‘rniga i ni oshiradi: j har doim 1 qoladi, j <= 3 sharti har doim to‘g‘ri, i cheksiz o‘sadi — ichki tsikl aylanib chiqadi (va tashqi hisoblagichni buzadi). Tuzatish: ichki sarlavhadagi j++:

```
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        cout << i * j << "\t";
    }
    cout << endl;
}
```

[Manba: $bug_{huntloops}$.txt, BUG 9]

L10. To‘rtburchakda 7 ni topish — listingning uzilishi [Hard]

Kutilgan: to‘rtburchakdagi BIRINCHI 7 topinib "First 7 found in row 0" chop etish. Mashqa faylidagi listing uzilgan (so‘zma-so‘z, barcha mavjud fragment):

```
int grid[3][3] = {
    {1, 7, 3},
```

L10. Listing uzilgan — bu masalada qanday saboq bor?

Javob. Masalaning to‘liq kodi kurs faylida mavjud emas (matn to‘rtburchak e’lonining boshlanishidan keyin darhol uziladi), shuning uchun so‘zma-so‘z tuzatish imkonsiz. Ko‘rinib turgan fragmentdan: 7 0-qator, 1-ustunda joylashgan — kutilgan chiqarish "First 7 found in row 0". Bu masala tekshiradigan qoida: break faqat eng ichki tsikldan chiqadi (5- dars: "break exits innermost loop or switch") — to‘rtburchak bo‘yicha qidirishda ustunlar bo‘yicha ichki tsikldan break qatorlar bo‘yicha tashqi tsiklni to‘xtatmaydi. Birinchi topilgandan keyin qidirni tugatish uchun "topildi" bayrog‘i yoki ichki bloklardan chiqish uchun goto ishlatiladi (6- dars: "goto provides a convenient way to exit from nested blocks"). Bu masalani kompilyatsiya qilinadigan fragment sifatida emas, balki qoidalar muhokamasi sifatida tahlil qiling.

[Manba: $bug_{huntloops}$.txt, BUG 10 — fragment asl faylda uzilgan]

6.3. Bug Hunt Pointers & Strings

"C BUG HUNT: POINTERS AND STRINGS" varaqi — o‘n ikkita masala. Barcha xatolarning kalibri gcc bilan kompilyatsiya qilish va AddressSanitizer/UBSan ostida ishlatish orqali tekshirilgan; ularning aksariyati belgilanmagan xulq-atvor, shuning uchun aniq alomatlar turli kompilyatorlar, bayroqlar va mashinalarda farq qilishi mumkin. O‘qituvchi kalibi ($bug_{huntpointersstringsanswerkey}$.txt) tuzatishlar uchun manba hisoblanadi; pastda har bir masala: listing so‘zma-so‘z, kalibdagi tushuntirish va tuzatilgan kod.

P1. Nol-terminalator yo‘q

```
char s[5] = {'h', 'e', 'l', 'l', 'o'};
printf("%s\n", s);
```

P1. "hello" kutilgan — bu kod nima chop etadi va nima uchun?

Javob. s da 5 ta xona bor, 5 ta harf uchun — '\0' uchun joy qolmagan. printf("%s") massivdan keyingi xotirani nolli baytga duch kelguncha o‘qishda davom etadi: chiqarish noaniq. Kalit bo‘yicha tuzatish: char s[6] = {'h','e','l','l','o','\0'}; yoki char s[] = "hello"; (kompilyator o‘zi terminator va o‘lchamni qo‘shib qo‘yadi):

```
char s[6] = {'h', 'e', 'l', 'l', 'o', '\0'};
printf("%s\n", s);

/* yoki: */
char s2[] = "hello";
printf("%s\n", s2);
```

[Manba: $bug_{huntpointersstrings}$.txt, BUG 1 + kalit]

P2. sizeof ko‘rsatkichda

```
char *s = "hello";
printf("%zu\n", sizeof(s));
```

P2. Satdning uzunligi 5 kutilgan — sizeof(s) qaytaradi?

Javob. s — bu char *, shuning uchun sizeof(s) — bu o‘z-o‘zidan ko‘rsatgichning o‘lchami (64-bitli mashinada 8), satdning uzunligi emas: 8 chiqariladi. Satdning uzunligini strlen(s) qaytaradi. Kalit maslahati: haqiqiy massivda sizeof (char a[] = "hello") 6 beradi, '\0' ni hisobga olgan holda.

```
char *s = "hello";
printf("%zu\n", strlen(s));
```

[Manba: bug$_{huntpointersstrings}$.txt, BUG 2 + kalit]

P3. Satdlarni == orqali solishtirish

```
char a[] = "cat";
char b[] = "cat";
if (a == b)
    puts("Same");
else
    puts("Different");
```

P3. Nima uchun "Different" chiqadi, matn bir xil?

Javob. == operatori ikki massivning manzillarini solishtiradi — bu xotiradagi ikki xil ob’ekt, shuning uchun solishtirish noto‘g‘ri. Satdlarning mazmunini strcmp solishtiradi: if (strcmp(a, b) == 0) — "Same" chiqadi.

```
char a[] = "cat";
char b[] = "cat";
if (strcmp(a, b) == 0)
    puts("Same");
else
    puts("Different");
```

[Manba: bug$_{huntpointersstrings}$.txt, BUG 3 + kalit]

P4. Bufer strcpy uchun juda kichik

```
char dest[5];
strcpy(dest, "hello");
printf("%s\n", dest);
```

P4. "hello" ni nusxalash uchun dest da necha bayt kerak va bu yerda nima bo‘ladi?

Javob. "hello" 6 bayt talab qiladi: 5 ta harf + '\0', dest esa 5 sig‘imli — strcpy bitta baytni massivdan tashqariga yozadi (bufer ortiqcha yuklanishi). Tuzatish: char dest[6]; yoki kattaroq.

```
char dest[6];
strcpy(dest, "hello");
printf("%s\n", dest);
```

[Manba: bug$_{huntpointersstrings}$.txt, BUG 4 + kalit]

P5. Satdli literalni o‘zgartirish

```
char *s = "hello";
s[0] = 'J';
printf("%s\n", s);
```

P5. Nima uchun s[0] = 'J'; dasturni qulatadi?

Javob. "hello" — bu faqat o‘qish uchun xotiradagi literal; s faqat unga ishora qiladi va s[0] = 'J' yozish read-only xotirani o‘zgartirishga harakat qiladi — odatda segmentatsiya (segfault). Kalit bo‘yicha tuzatish: char s[] = "hello"; — shu tarzda o‘zgaruvchi nusxa massivda yaratiladi va dastur "Jello" chiqaradi.

```
char s[] = "hello";
s[0] = 'J';
printf("%s\n", s);
```

[Manba: bug$_{huntpointersstrings}$.txt, BUG 5 + kalit]

P6. Lokal o‘zgaruvchiga ko‘rsatgichni qaytarish

```
char *greeting(void) {
    char buf[20];
```

```
    strcpy(buf, "Hi there");
    return buf;
}

int main(void) {
    char *g = greeting();
    printf("%s\n", g);
    return 0;
}
```

P6. Nima uchun greeting() chaqirilgandan so‘ng g ko‘rsatgichi "osilib" qoladi?

Javob. buf greeting()ning stek kadrida yashaydi, u funksiya qaytgandan so‘ng darhol yo‘qoladi — g osib qolgan ko‘rsatgichga (dangling) aylanadi; *g o‘qish - noaniq xulq-atvor. Kompilyatorlar odatda ogohlantiradi. Kalit bo‘yicha tuzatishlar: statik-massiv, chaqiruvchi tomonidan berilgan bufer, yoki chaqiruvchi tomonidan bo‘shatiladigan malloc-xotira. Statik variant bilan:

```
char *greeting(void) {
    static char buf[20];
    strcpy(buf, "Hi there");
    return buf;
}

int main(void) {
    char *g = greeting();
    printf("%s\n", g);
    return 0;
}
```

[Manba: $bug_{huntpointersstrings}$.txt, BUG 6 + kalit]

P7. Initsizlangan (yovvoyi) ko‘rsatgich

```
int *p; // -> Memory address
*p = 10;
printf("%d\n", *p);
```

P7. *p = 10 qayerga yozadi va nima uchun bu xatarli, hatto 10 chiqsa ham?

Javob. p hech qachon berilmagan — u chiqindu manzilni o‘z ichiga oladi va *p = 10 noma’lum manzilga yozadi: dastur yiqilishi mumkin, yoki "ishlashi" mumkin - xatarli holat (kalit testida 10 tasodifiy ravishda chiqdi). Kalit bo‘yicha tuzatish - p ni haqiqiy xotiraga ishora qilish: int v; int* p = &v; yoki p = malloc(sizeof(int));

```
int v;
int *p = &v;
*p = 10;
printf("%d\n", *p);
```

[Manba: $bug_{huntpointersstrings}$.txt, BUG 7 + kalit]

P8. Qiymat bo‘yicha uzatish

```
void swap(int a, int b) {
    int t = a;
    a = b;
    b = t;
}

int main(void) {
    int x = 1, y = 2;
    swap(x, y);
    printf("%d %d\n", x, y);
    return 0;
```

```
}
```

P8. Nima uchun swap(x, y) dan keyin "1 2" chiqadi, "2 1" emas?

Javob. C nusxalarni uzatadi: swap o‘zining xususiy lokal nusxalarini x va y ni almashtiradi, main o‘zgaruvchulari o‘zgarmaydi. Kalit bo‘yicha tuzatish - 7-ma’ruzadagi ko‘rsatgichli bir xil funksiya: void swap(int *a, int* b) va chaqirish swap(&x, &y):

```
void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}

int main(void) {
    int x = 1, y = 2;
    swap(&x;, &y;);
    printf("%d %d\n", x, y);
    return 0;
}
```

[Manba: $bug_{huntpointersstrings}$.txt, BUG 8 + kalit]

P9. malloc '\0' uchun +1siz

```
char *s = "hello";
char *copy = malloc(strlen(s));
strcpy(copy, s);
printf("%s\n", copy);
free(copy);
```

P9. Heap-blok necha bayt ortiqcha yuklanadi va nima uchun?

Javob. strlen terminatormi hisoblamaydi: "hello" uchun malloc(strlen(s)) 5 bayt ajratadi, strcpy esa 6 yozadi (5 ta harf + '\0') — heap-blok bitta bayt ortiqcha yuklanadi. Kalit bo‘yicha tuzatish: malloc(strlen(s) + 1).

```
char *s = "hello";
char *copy = malloc(strlen(s) + 1);
strcpy(copy, s);
printf("%s\n", copy);
free(copy);
```

[Manba: $bug_{huntpointersstrings}$.txt, BUG 9 + kalit]

P10. Tengsiz hisoblagich teskari siklda

```
char s[] = "abc";
for (size_t i = strlen(s) - 1; i >= 0; i--) {
    putchar(s[i]);
}
putchar('\n');
```

P10. Nima uchun "teskari" sikl 'a' dan keyin to‘xtamaydi?

Javob. $size_t$ - belgili emas tur: shart i >= 0 har doim to‘g‘ri. i = 0 dan keyin kamaytish i ni katta songa aylantiradi va s[i] massivdan juda uzoqda o‘qiydi. Kalitdan ikkita tuzatish: belgili hisoblagich yoki i > 0 sharti bilan sikl va s[i - 1] ga kirish:

```
char s[] = "abc";
    for (int i = (int)strlen(s) - 1; i >= 0; i--) {
        putchar(s[i]);
    }
    putchar('\n');
```

```
/* yoki: */
char t[] = "abc";
for (size_t i = strlen(t); i > 0; i--) {
    putchar(t[i - 1]);
}
putchar('\n');
```

[Manba: bug$_{huntpointersstrings}$.txt, BUG 10 + kalit]

P11. scanf da & etishmayapti

```
int x;
scanf("%d", x);
printf("%d\n", x);
```

P11. scanf("%d", x) &siz nima qiladi?

Javob. scanf o‘zgaruvchi manzilini talab qiladi; x (int bo‘lmagan) ni uzatish scanf ni uni chiqindilar qiymati sifatida ko‘rsatgich sifatida talqin qilishiga va shu manzilga yozishiga olib keladi - bu belgilanmagan xulq; ko‘p kompilyatorlar -Wall bilan ogohlantiradi. Tuzatish: scanf("%d", &x);.

```
int x;
scanf("%d", &x;);
printf("%d\n", x);
```

[Manba: bug$_{huntpointersstrings}$.txt, BUG 11 + kalit]

P12. free dan keyin foydalanish

```
int *p = malloc(sizeof(int));
*p = 42;
free(p);
printf("%d\n", *p);
```

P12. Nima uchun free(p) dan keyin *p ni choplash xato?

Javob. free(p) dan keyin xot allaqachon qayta ishlatilgan bo‘lishi mumkin: *p ni o‘qish 42, chiqindilar yoki tushib ketishi mumkin (free dan keyin foydalanish - belgilanmagan xulq). Kalit bo‘yicha: free dan keyin* p ni tegmang; chiqarishdan oldin choplang; free dan keyin p = NULL deb belgilang, xato aniq bo‘lsin:

```
int *p = malloc(sizeof(int));
*p = 42;
printf("%d\n", *p);
free(p);
p = NULL;
```

[Manba: bug$_{huntpointersstrings}$.txt, BUG 12 + kalit]

Xatolar xaritasi (kalitdan)

Xato turi	Masala raqamlari
Nol-terminalator etishmaydi / +1 uchun '\0'	1, 4, 9
Ko‘rsatgich va massiv chalkashligi	2, 3
Faqat o‘qish uchun xotira / osilib qolgan xotira	5, 6, 12
Dastlabki belgilanmagan ko‘rsatgichlar	7, 11
Qiymat bo‘yicha uzatish	8
Belgisay aylanish (wraparound)	10

Xato turi bo‘yicha tez tartib [Manba: bug$_{huntpointersstringsanswerkey}$.txt]

7. Yordamchi: asosiy qoidalar va jadvallar

Oxirgi bob - test va imtihondan oldin barcha kerakli narsalar: operator ustunligi, printf/scanf formatlarining to‘liq jadvali, 4-7 haftalardagi C asosiy qoidalari bir sahifada, o‘ndan javob bir qatorda va kurs bo‘yicha keyingi qadam.

7.1. Operator ustunligi

Ustunlik	Operatorlar	Izoh
1 (eng yuqori)	++ – –, (cast), sizeof	birlik operatorlari, tur o‘tkazish, hajm
2	*, /, %	ko‘paytiruvchilar - + va – dan yuqori
3	+, –	qo‘shimcha
4	==, !=	tenglikni solishtirish - && va || dan yuqori
5	&&, ||	mantiqiy
6 (eng past)	=, +=, –=, *=, /=, %=	tayinlash - juda past ustunlik

Ustunlik va hisoblash tartibi [Manba: lecture$_4$, slayd 25]

Slayd qoidasi: noaniqlik va operator ustunligining yon ta’siridan qochish uchun qavs ( ) juda ko‘p ishlating. Tenglik misollari:

```
y = x*3+2                 /* same as y = (x*3)+2 */
x != 0 && y == 0          /* same as (x != 0) && (y == 0) */
d = c >= '0' && c <= '9'  /* same as d = (c >= '0') && (c <= '9') */
```

7.2. printf/scanf formatlari

To‘liq format belgisi: **%[flags][width][.precision][length]<type>**. Tur belgichlari (6 va 7-ma’ruzalar):

Belgi	Tur	Misol / chiqarish
d, i	int	printf("%d",10); → 10
u	unsigned int	printf("%u",10); → 10
o	unsigned int (sakkizlik)	7-ma’ruzadan: u, o, x, X — unsigned int
x, X	unsigned int (o‘nlik)	printf("%x",10); → 0xa
c	char	printf("%c",'A'); → A
s	string	printf("%s","hello"); → hello
f	float/double	printf("%f",2.3); → 2.3
e, E	haqiqiy (eksponenta)	1e3, 1.2E3, 1E–3
F, g, G	double	7-ma’ruzadan: e, E, f, F, g, G — double
%	% belgisi	printf("%d %%",10); → 10%

printf tur belgichlari [Manba: Lecture$_6$, slayd 12; Lecture$_{71}$, slayd 3]

Modifikator	Ma’no
h	short sifatida talqin qilish; i, d, o, u, x bilan

Modifikator	Ma’no
l	long sifatida talqin qilish; i, d, o, u, x bilan
L	double sifatida talqin qilish; e, f, g bilan

Uzunlik modifikatorlari [Manba: Lecture$_6$, slayd 16]

printf chaqiruvi	Chiqarish
printf("%d",10)	«10»
printf("%4d",10)	«bb10» (b - bo‘shliq; width)
printf("%7s","hello")	«bbhello»
printf("%d,%+d,%+d",10,-10)	«10,+10,-10» (bayroq +)
printf("%04d",10)	«0010» (bayroq 0)
printf("%-7s","hello")	«hellobb» (bayroq –: chap tomonga)
printf("%.2f,%.0f",1.141,1.141)	«1.14,1» (precision)
printf("%.2e,%.0e",1.141,100.00)	«1.14e+00,1e+02»
printf("%.4s","hello")	«hell»
printf("%.1s","hello")	«h»

width, flags, precision [Manba: Lecture$_6$, slaydlar 13–15]

scanf - printf ning kirish analogi, bir xil format bilan; bo‘sh belgilarni e’tiborsiz qoldiradi; o‘qilgan elementlar sonini yoki EOF qaytaradi; **argumentlar - o‘zgaruvchi manzillari (massiv nomlari bundan mustasno)**:

printf	scanf
printf("%d",x)	scanf("%d",&x)
printf("%f",f)	scanf("%f",&f)
printf("%s",str)	scanf("%s",str) - & kerak emas
printf("%s",str)	scanf("%20s",str) - & kerak emas
printf("%s %s",fname,lname)	scanf("%20s %20s",fname,lname)

printf ↔ scanf [Manba: Lecture$_6$, slayd 20]

stdio.h belgili funktsiyalar: putchar(c) - belgini standart chiqarishga (belgi yoki EOF qaytaradi); getchar() - standart kirishdagi keyingi belgi yoki EOF; getc(fp)/putc(c, fp) - oqim uchun xuddi shu; getchar() - bu getc(stdin), putchar(c) - putc(c, stdout). Satr bo‘yicha: fgets(line, maxlen, fp) satrni o‘qiydi (shu jumladan satr o‘tkazuvchini), fputs(line, fp) yozadi; fayllar: fopen(name, mode) "r", "w", "a" rejimlari bilan ("b" ikkilik uchun), fclose(fp); stdin/stdout/stderr - hammasi FILE*.

7.3. C asosiy qoidalari

- **Operand turlari bo‘linishni hal qiladi**: 4/3 → 1 (int), 4.0/3.0 → 1.333333; qabul qiluvchi turi (double fraction = 4/3) ta’sir qilmaydi — bo‘linish tayinlanishgacha amalga oshiriladi.
- **O‘zgaruvchi nomlari**: harflar, raqamlar va _; harf bilan boshlanadi; kalit so‘zlardan foydalab bo‘lmaydi; katta-kichik harflar muhim (x va X — turli o‘zgaruvchilar).
- **O‘lchamlar kafolatlari**: sizeof(char) == 1; sizeof(char) < sizeof(short) <= sizeof(int) <= sizeof(long); unsigned taxminan ikki barobar katta diapazonga ega; float aniqligi — 6, double — 15 muhim raqam.

- **Haqiqat va yolg‘on**: har qanday nol bo‘lmagan qiymat — haqiqat, 0 — yolg‘on; shart — sonli ifoda yoki ko‘rsatkich; C99 da bool — <stdbool.h> dan.
- **== — solishtirish, = — tayinlash**; float/double uchun solishtirish == chekli aniqlik tufayli ishonchli emas (0.1 aniq ifodalab bo‘lmaydi).
- **Qisqa tutashuv**: && va || natija aniq bo‘lganda hisoblashni to‘xtatadi; o‘ng tomonning yon effektlari bajarilmay qolishi mumkin.
- **break** eng ichki sikl yoki switch dan chiqadi; **case da break bo‘lmaganda bajarish keyingi case ga o‘tadi** yangi tekshirishsiz.
- **continue** ichki sikl tanasining qolgan qismini o‘tkazib yuborib, shartga o‘tadi (for da — yangilanishga).
- **for(...) dan keyingi nuqta-vergul** sikl tanasini bo‘sh qiladi — uning keyingi blok bir marta bajariladi.
- **Argumentlar qiymati bo‘yicha uzatiladi**: funksiya chaqiruvchi o‘zgaruvchini o‘zgartirishi uchun ko‘rsatgichni uzating (swap(&x, &y)); qaytarilgan tuzilma — alohida nusxa, uni chaqiruvchi qayerda saqlashni hal qiladi.
- **C satiri — char massiv '\0' bilan**: strlen terminatorni hisoblamaydi; nusxalash uchun +1 bayt kerak (malloc(strlen(s) + 1)); %s nol baytgacha o‘qiydi.
- **Satrlar konstantalari faqat o‘qish uchun**: char *s = "hello"; s[0] = 'J'; — segfault; o‘zgartiriladigan nusxa — char s[] = "hello";.
- **scanf manzillari kerak**: scanf("%d", &x); char massivi nomi allaqachon manzil: scanf("%s", str).
- **Tayinlanmagan o‘zgaruvchilar — chiqindilar**: lokal double discount qiymatisiz — undefined behavior; C da standart qiymatlar yo‘q.
- **Rekursiya = asosiy holat + kichik vazifa + progress**: progresssiz ($count_{down}(n)$ o‘rniga $count_{down}(n - 1)$) — cheksiz rekursiya va stek ortib ketishi.

7.4. Bir qatorda o‘n javob

- 1. double fraction = 4/3 ni %f bilan nima chiqaradi? — 1.000000: butun sonli bo‘linish 1 beradi, konvertatsiyagacha.
- 2. float va double da nechta muhim raqam bor? — 6 va 15 (FLT_{DIG}, DBL_{DIG}).
- 3. Kabisa yili? — year % 400 == 0 || (year % 4 == 0 && year % 100 != 0): 2000 — ha, 1900 va 2100 — yo‘q.
- 4. switch da break bo‘lmaydi nima bo‘ladi? — keyingi case ga o‘tadi tekshirishsiz ('A#' melodiyasi C ni, keyin D ni chiqaradi).
- 5. do-while necha marta bajariladi? — kamida bir marta: shart tanadan keyin tekshiriladi.
- 6. continue nima qiladi? — sikl tanasining qolgan qismini o‘tkazib yuborib, shartga o‘tadi.
- 7. main da x va y ni qanday almashtirish mumkin? — void swap(int *a, int* b) va chaqirish swap(&x, &y): qiymati bo‘yicha hech narsa chiqmaydi.
- 8. strcmp("aA", "aA") qaytaradi? — 0: satrlar teng; >0 — str1 katta, <0 — kichik.
- 9. $count_{down}(3)$ va $count_{up}(3)$ nima chiqaradi? — 3 2 1 va 1 2 3: chaqirishgacha chiqarish — tushishda, keyin — qaytishda.
- 10. fibonacci(4)? — 3; ikkita asosiy holat (F(0)=0, F(1)=1), fibonacci(2) ikki marta hisoblanadi.

7.5. Keyin nima

7-haftadan keygi ma’ruzalar materiallari bo‘yicha kurs to‘g‘ridan-to‘g‘ri slaydlarda e’lon qilingan mavzular bilan davom etadi: «More searching next week…» (ma’ruza 7, ikkilik qidirish haqidagi slayd — qidirish haqida

ko‘proq), «More about sizeof() next time…» (sizeof haqidagi slayd) va «Implicit cast to/from void * also possible (more next week…)» (ko‘rsatkichlarni aylantirish haqidagi slayd), shuningdek, 5-ma’ruzadagi modullar tushunchasi tashqi kutxonalarga ko‘chiriladi («Same concept carries over to external libraries (next week…)»). Midtermdan oldi ushbu qo‘llanmada 4–7 haftalarni quyidagi tartibda takrorlang: turlar va bo‘linish (2-bob) → shartlar, switch va sikllar (3-bob) → funksiyalar, tuzilmalar va kirish-chiqarish (4-bob) → ko‘rsatkichlar, satrlar va rekursiya (5-bob), keyin barcha bug hunt-larni (6-bob) va bu shpalgalni o‘tkazing: 4-haftaning uchta chiqish tekshiruvi (nima uchun 4/3 kasrni yo‘qotadi; nima uchun 2100 — oddiy yil; nima uchun chegaraviy birinchi keladi) va 7-haftaning uchta chiqish tekshiruvi (faktorial qayerda to‘xtaydi; nima uchun n – 1 asosiy holatga intiladi; factorial(3) nima kutadi) — o‘zingizni tekshirish uchun tayyor savollar.